Nutan Gupta & Mathias Helseth

Master Thesis, University of Inland Norway

Appendix B

Source Code for Data Cleaning, Feature Engineering and Master Dataset File Creation

```
# ============================================================
# Purpose: Clean all raw data files, merge, engineer features,
#          and produce master_clean_NO1.csv for modelling
#
# Input files:
#   - norway_day_ahead_prices_long.csv
#   - hydro_reserves_clean.csv
#   - water_inflow.csv
#   - nordic_system_forwards_clean.csv
#   - nordic_epads_clean.csv
#   - TTF_Daily_.csv
#   - EEX_EUA_Daily_Continuous_2012_2025.csv
#   - brent_front_month_m1.csv
#   - norway_weather_datasetunchanged.csv
#   - load_NO1_raw.csv
#   - eur_nok_clean.csv
#
# Output files:
#   - master_dataset_NO1.csv   (2881 rows - with NaN warmup)
#   - master_clean_NO1.csv     (2516 rows - model ready)
# ============================================================

import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')
from pathlib import Path
from functools import reduce

# - SET YOUR DATA FOLDER --------------------------
DATA_FOLDER = Path('/RECENT DATA')

# Study period start - determined by earliest TTF gas data
STUDY_START = '2017-11-24'


# ============================================================
```

```python
# HELPER FUNCTIONS
# ============================================================

def to_daily_ffill(df, date_col='date', start=STUDY_START):
    """Resample any dataframe to daily frequency using forward-fill."""
    df = df.set_index(date_col).resample('D').ffill().reset_index()
    df = df[df[date_col] >= start].reset_index(drop=True)
    return df

def assert_clean(df, name):
    """Assert no nulls and no inf values in a dataframe."""
    nulls = df.isnull().sum().sum()
    infs  = np.isinf(df.select_dtypes(include=np.number)).sum().sum()
    assert nulls == 0, f"{name}: {nulls} null values found!"
    assert infs  == 0, f"{name}: {infs} inf values found!"
    print(f"  {name:<40}: shape={df.shape}  "
          f"date={df.iloc[:,0].min().date()} → {df.iloc[:,0].max().date()}")


# ============================================================
# SHARED FILES (same for all 5 zones)
# ============================================================
print("=" * 60)
print("SECTION 1 - CLEANING SHARED FILES")
print("=" * 60)

# - FILE 1: HYDRO RESERVOIR LEVELS ----------------------
# Source: NVE via hydro_reserves_clean.csv
# Raw   : Weekly, multiple zones (NO, NO1-NO5)
# Keep  : National NO level only
# Action: Weekly → daily via forward-fill
print("\n--- File 1: Hydro Reservoir ---")

df_hydro = pd.read_csv(DATA_FOLDER / 'hydro_reserves_clean.csv')
df_hydro['date'] = pd.to_datetime(df_hydro['datetime'])

# Keep national level only
df_hydro = df_hydro[df_hydro['zone'] == 'NO'][['date','hydro_reserve_gwh']].copy()

# Weekly → daily forward-fill
df_hydro = to_daily_ffill(df_hydro)
df_hydro['hydro_reserve_gwh'] = df_hydro['hydro_reserve_gwh'].ffill().bfill()

assert_clean(df_hydro, "hydro_reserves")


# - FILE 2: WATER INFLOW AND SNOWPACK --------------------
# Source: NVE via water_inflow.csv
# Raw   : Weekly, Norwegian column names, data from 1958 onwards
# Keep  : National NO level only
# Action: Translate columns, convert year+week → date, weekly → daily
print("\n--- File 2: Water Inflow ---")

df_inflow = pd.read_csv(DATA_FOLDER / 'water_inflow.csv', sep=';',
encoding='utf-8')
```

```
93
94  # Rename Norwegian columns to English
95  df_inflow = df_inflow.rename(columns={
96      'Område' : 'zone',
97      'År'     : 'year',
98      'Uke'    : 'week',
99      'Nyttbart tilsig HBV (brukes før 2015 og til min/maks/gj.snitt)' :
100     'inflow_hbv_gwh',
101     'Nyttbart tilsig (produksjonsdata og magasinstatistikk, brukes f.o.m. 2015)' :
102     'inflow_prod_gwh',
103     'Magasinavvik' : 'reservoir_deviation_gwh',
104     'Snø'          : 'inflow_snow_gwh',
105 })
106
107 # Keep national NO level, drop metadata rows
108 df_inflow = df_inflow[df_inflow['zone'] == 'NO'][
109     ['year', 'week', 'inflow_hbv_gwh', 'inflow_snow_gwh',
110     'reservoir_deviation_gwh']
111 ].copy()
112
113 # Convert to numeric - removes any non-numeric metadata rows
114 df_inflow['year'] = pd.to_numeric(df_inflow['year'], errors='coerce')
115 df_inflow['week'] = pd.to_numeric(df_inflow['week'], errors='coerce')
116 df_inflow = df_inflow.dropna(subset=['year', 'week'])
117
118 # Convert ISO year + week number → calendar date (Monday of that week)
119 df_inflow['date'] = pd.to_datetime(
120     df_inflow['year'].astype(int).astype(str)
121     + '-W'
122     + df_inflow['week'].astype(int).astype(str).str.zfill(2)
123     + '-1',
124     format='%G-W%V-%u'
125 )
126
127 # Keep only the columns we need, sort by date
128 df_inflow = (df_inflow[['date', 'inflow_hbv_gwh', 'inflow_snow_gwh',
129 'reservoir_deviation_gwh']]
130              .sort_values('date')
131              .reset_index(drop=True))
132
133 # Weekly → daily forward-fill
134 df_inflow = to_daily_ffill(df_inflow)
135
136 # Convert value columns to numeric and fill any remaining nulls
137 for col in ['inflow_hbv_gwh', 'inflow_snow_gwh', 'reservoir_deviation_gwh']:
138     df_inflow[col] = pd.to_numeric(df_inflow[col], errors='coerce').ffill()
139     .bfill()
140
141 assert_clean(df_inflow, "water_inflow")
142
143 # Store full history for z-score calculation later
144 # (use full 1958-2026 record, not just study period)
145 inflow_hist = pd.read_csv(DATA_FOLDER / 'water_inflow.csv', sep=';',
146 encoding='utf-8')
147 inflow_hist = inflow_hist.rename(columns={
```

```
    'Område' : 'zone',
    'Uke'    : 'week',
    'Nyttbart tilsig HBV (brukes før 2015 og til min/maks/gj.snitt)' :
    'inflow_hbv_gwh',
})
inflow_hist = inflow_hist[inflow_hist['zone'] == 'NO'][['week',
'inflow_hbv_gwh']].dropna()
inflow_hist['week']          = pd.to_numeric(inflow_hist['week'],
errors='coerce')
inflow_hist['inflow_hbv_gwh'] = pd.to_numeric(inflow_hist['inflow_hbv_gwh'],
errors='coerce')
inflow_hist = inflow_hist.dropna()
weekly_norm = inflow_hist.groupby('week')['inflow_hbv_gwh'].mean()
weekly_std  = inflow_hist.groupby('week')['inflow_hbv_gwh'].std().replace(0,
np.nan)
print(f"  Inflow historical norm: {len(inflow_hist)} weekly obs used (full
1958-2026 record)")


# - FILE 3: NORDIC SYSTEM FORWARD PRICES -------------------
# Source: Nordic exchange via nordic_system_forwards_clean.csv
# Raw   : Daily, multiple tenors (M1-M6, Q1-Q8, Y1-Y10), SYSTEM zone
# Keep  : M1 and Y1 only (M2/M3/Q1 have r>0.89 with M1 - redundant)
# Action: Pivot to wide format, daily forward-fill weekends
print("\n--- File 3: System Forward Prices ---")

df_fwd = pd.read_csv(DATA_FOLDER / 'nordic_system_forwards_clean.csv')
df_fwd['date'] = pd.to_datetime(df_fwd['datetime'])

# Keep M1 and Y1 only
df_fwd = df_fwd[
    ((df_fwd['tenor_type'] == 'M') & (df_fwd['tenor_num'] == 1)) |
    ((df_fwd['tenor_type'] == 'Y') & (df_fwd['tenor_num'] == 1))
].copy()

# Remove duplicates - two contract codes for same tenor
df_fwd = df_fwd.drop_duplicates(subset=['date', 'tenor_type', 'tenor_num'])

# Pivot: one row per date, one column per tenor
pivot_fwd = df_fwd.pivot_table(
    index   = 'date',
    columns = ['tenor_type', 'tenor_num'],
    values  = 'settlement'
)
pivot_fwd.columns = [f"forward_{t}{n}" for t, n in pivot_fwd.columns]
df_fwd_clean = pivot_fwd.reset_index()

# Daily forward-fill weekends
df_fwd_clean = to_daily_ffill(df_fwd_clean)
for col in ['forward_M1', 'forward_Y1']:
    df_fwd_clean[col] = df_fwd_clean[col].ffill().bfill()

assert_clean(df_fwd_clean, "system_forwards")
```

```
# - FILE 4: TTF NATURAL GAS PRICES ----------------------
# Source: PEGAS/ICE via TTF_Daily_.csv
# Raw   : Semicolon-delimited, UTF-8 BOM header, European decimal notation
# Action: Parse format, extract settlement price, daily forward-fill
print("\n--- File 4: TTF Natural Gas ---")

df_ttf = pd.read_csv(DATA_FOLDER / 'TTF_Daily_.csv', sep=';', encoding='utf-8-sig')

# Rename columns (first column contains BOM artifact #NAME?)
df_ttf.columns = [
    'drop', 'Open', 'High', 'Low', 'Settlement',
    'Close', 'Volume', 'OI', 'Date', 'ContractName'
]

# Parse date
df_ttf['date'] = pd.to_datetime(df_ttf['Date'], format='%d/%m/%Y %H:%M',
errors='coerce')

# Parse settlement - replace European decimal comma with period
df_ttf['fuel_gas_eur_mwh'] = (df_ttf['Settlement']
                              .astype(str)
                              .str.replace(',', '.', regex=False)
                              .apply(pd.to_numeric, errors='coerce'))

df_ttf = (df_ttf[['date', 'fuel_gas_eur_mwh']]
          .dropna()
          .sort_values('date')
          .reset_index(drop=True))

# Daily forward-fill weekends
df_ttf = to_daily_ffill(df_ttf)
df_ttf['fuel_gas_eur_mwh'] = df_ttf['fuel_gas_eur_mwh'].ffill().bfill()

assert_clean(df_ttf, "TTF_gas")


# - FILE 5: EU CARBON ALLOWANCE PRICE ---------------------
# Source: EEX via EEX_EUA_Daily_Continuous_2012_2025.csv
# Raw   : Daily, no gaps, already clean
# Action: Rename, parse date, resample for consistency
print("\n--- File 5: EU Carbon Price ---")

df_carbon = pd.read_csv(DATA_FOLDER / 'EEX_EUA_Daily_Continuous_2012_2025.csv')
df_carbon = df_carbon.rename(columns={
    'Date'             : 'date',
    'AuctionPrice_EUR' : 'fuel_carbon_eur_t'
})
df_carbon['date'] = pd.to_datetime(df_carbon['date'])
df_carbon = to_daily_ffill(df_carbon)
df_carbon['fuel_carbon_eur_t'] = df_carbon['fuel_carbon_eur_t'].ffill().bfill()

assert_clean(df_carbon, "EUA_carbon")


# - FILE 6: BRENT CRUDE OIL PRICE -----------------------
```

```
# Source: ICE via brent_front_month_m1.csv
# Raw   : Trading days only, dd/mm/yyyy date format
# Action: Parse dayfirst format, daily forward-fill weekends
print("\n--- File 6: Brent Crude Oil ---")

df_brent = pd.read_csv(DATA_FOLDER / 'brent_front_month_m1.csv')
df_brent['date'] = pd.to_datetime(df_brent['datetime'], dayfirst=True)
df_brent = df_brent.rename(columns={'brent_close': 'fuel_brent_usd_bbl'})[
    ['date', 'fuel_brent_usd_bbl']
]
df_brent = to_daily_ffill(df_brent)
df_brent['fuel_brent_usd_bbl'] = df_brent['fuel_brent_usd_bbl'].ffill().bfill()

assert_clean(df_brent, "Brent_crude")


# - FILE 7: EUR/NOK EXCHANGE RATE -----------------------
# Source: Central bank data via eur_nok_clean.csv
# Raw   : Trading days only
# Action: Daily forward-fill weekends
print("\n--- File 7: EUR/NOK Exchange Rate ---")

df_fx = pd.read_csv(DATA_FOLDER / 'eur_nok_clean.csv')
df_fx['date'] = pd.to_datetime(df_fx['datetime'])
df_fx = df_fx.rename(columns={'eur_nok': 'macro_eur_nok'})[['date',
'macro_eur_nok']]
df_fx = to_daily_ffill(df_fx)
df_fx['macro_eur_nok'] = df_fx['macro_eur_nok'].ffill().bfill()

assert_clean(df_fx, "EUR_NOK")


# - MERGE ALL SHARED FILES --------------------------
print("\n--- Merging shared files ---")
shared_files = [df_hydro, df_inflow, df_fwd_clean, df_ttf, df_carbon, df_brent,
df_fx]
df_shared = reduce(
    lambda left, right: pd.merge(left, right, on='date', how='inner'),
    shared_files
)
df_shared = df_shared.sort_values('date').reset_index(drop=True)
assert_clean(df_shared, "SHARED_MASTER")


# ============================================================
# ZONE-SPECIFIC FILES FOR NO1
# ============================================================
print("\n" + "=" * 60)
print("SECTION 2 - CLEANING ZONE-SPECIFIC FILES (NO1)")
print("=" * 60)

# - FILE 8: DAY-AHEAD SPOT PRICES - NO1 (TARGET VARIABLE) ----------
# Source: NordPool via norway_day_ahead_prices_long.csv
# Raw   : Hourly, all 5 zones (NO1-NO5), format dd/mm/yyyy HH:MM
# Keep  : NO1 only
```

```
# Action: Aggregate 24 hourly prices → 1 daily mean
print("\n--- File 8: Spot Prices (NO1 target) ---")

df_spot = pd.read_csv(DATA_FOLDER / 'norway_day_ahead_prices_long.csv')
df_spot['datetime'] = pd.to_datetime(df_spot['datetime'], format='%d/%m/%Y %H:%M')
df_spot['date']     = df_spot['datetime'].dt.floor('D')

# Keep NO1 zone only
df_spot = df_spot[df_spot['zone'] == 'NO1'].copy()

# Aggregate hourly → daily mean
# Note: DST days have 23 or 25 hours - mean handles this correctly
df_spot = (df_spot
           .groupby('date')['price']
           .mean()
           .reset_index()
           .rename(columns={'price': 'spot_price'}))

df_spot = df_spot[df_spot['date'] >= pd.Timestamp(STUDY_START)].
reset_index(drop=True)

# Negative prices are KEPT - valid market phenomenon (excess supply)
# Extreme prices (up to 660 EUR/MWh) are KEPT - 2021-2022 energy crisis
print(f"  Spot_prices (NO1): shape={df_spot.shape}  "
      f"min={df_spot['spot_price'].min():.2f}  max={df_spot['spot_price'].max()
      :.2f}  "
      f"negative_days={(df_spot['spot_price'] < 0).sum()}")


# - FILE 9: AREA PRICE DIFFERENTIALS (EPAD) - NO1 --------------
# Source: Nordic exchange via nordic_epads_clean.csv
# Raw   : Multiple zones, multiple tenors
# Keep  : Oslo zone (SYOSLAFUTBL = NO1), M1 and Y1 only
#         (Q1 excluded: r=0.97 with M1 - near-duplicate)
print("\n--- File 9: NO1 EPADs ---")

df_epad = pd.read_csv(DATA_FOLDER / 'nordic_epads_clean.csv')
df_epad['date'] = pd.to_datetime(df_epad['datetime'], dayfirst=True,
errors='coerce')

# Keep Oslo zone only (= NO1)
df_epad = df_epad[df_epad['zone'] == 'SYOSLAFUTBL'].copy()

# Keep M1 and Y1 tenors only
df_epad = df_epad[
    ((df_epad['tenor_type'] == 'M') & (df_epad['tenor_num'] == 1)) |
    ((df_epad['tenor_type'] == 'Y') & (df_epad['tenor_num'] == 1))
].copy()

# Remove duplicates
df_epad = df_epad.drop_duplicates(subset=['date', 'tenor_type', 'tenor_num'])

# Pivot to wide format
pivot_epad = df_epad.pivot_table(
    index   = 'date',
```

```
    columns = ['tenor_type', 'tenor_num'],
    values  = 'settlement'
)
pivot_epad.columns = [f"epad_{t}{n}" for t, n in pivot_epad.columns]
df_epad_clean = pivot_epad.reset_index()

# Daily forward-fill weekends
df_epad_clean = to_daily_ffill(df_epad_clean)
for col in ['epad_M1', 'epad_Y1']:
    df_epad_clean[col] = df_epad_clean[col].ffill().bfill()

assert_clean(df_epad_clean, "EPAD_NO1")


# - FILE 10: ELECTRICITY LOAD - NO1 ---------------------
# Source: ENTSO-E via load_NO1_raw.csv
# Raw   : Hourly, UTC timestamps with timezone offset
# Action: Convert UTC → Oslo local time, aggregate hourly → daily mean
print("\n--- File 10: Electricity Load (NO1) ---")

df_load = pd.read_csv(DATA_FOLDER / 'load_NO1_raw.csv')

# Parse UTC timestamps
df_load['datetime'] = pd.to_datetime(df_load['datetime'], utc=True)

# Convert to Oslo local time (handles CET/CEST DST automatically)
df_load['date'] = (df_load['datetime']
                   .dt.tz_convert('Europe/Oslo')
                   .dt.floor('D')
                   .dt.tz_localize(None))

# Aggregate hourly → daily mean
# DST transition days (23h or 25h) are handled correctly by mean
df_load = (df_load
           .groupby('date')['Actual Load']
           .mean()
           .reset_index()
           .rename(columns={'Actual Load': 'load_mw'}))

df_load = df_load[df_load['date'] >= pd.Timestamp(STUDY_START)]
.reset_index(drop=True)
print(f"  Load_NO1: shape={df_load.shape}  "
      f"mean={df_load['load_mw'].mean():.0f} MW")


# - FILE 11: WEATHER DATA - NO1 ------------------------
# Source: MET Norway via norway_weather_datasetunchanged.csv
# Raw   : All 5 zones, 14 columns (7 completely empty)
# Keep  : NO1 zone, temp/wind/precip only
# Drop  : air_pressure, snow_depth, relative_humidity, cloud_area_fraction,
#         dew_point, shortwave_radiation, wind_speed_of_gust (all 100% empty)
print("\n--- File 11: Weather Data (NO1) ---")

df_wx = pd.read_csv(DATA_FOLDER / 'norway_weather_datasetunchanged.csv')
df_wx['date'] = pd.to_datetime(df_wx['date'])
```

```
423
424 # Keep NO1 zone, keep only 3 non-empty columns
425 df_wx = df_wx[df_wx['zone'] == 'NO1'][['date', 'temp', 'wind', 'precip']].copy()
426
427 # Rename clearly
428 df_wx = df_wx.rename(columns={
429     'temp'   : 'weather_temp_c',
430     'wind'   : 'weather_wind_ms',
431     'precip' : 'weather_precip_mm',
432 })
433
434 df_wx = df_wx[df_wx['date'] >= pd.Timestamp(STUDY_START)].reset_index(drop=True)
435
436 # Fill minor nulls
437 for col in ['weather_temp_c', 'weather_wind_ms', 'weather_precip_mm']:
438     df_wx[col] = df_wx[col].ffill().bfill()
439
440 # Negative temperatures are KEPT - valid Norwegian winter values
441 print(f"  Weather_NO1: shape={df_wx.shape}  "
442       f"temp_min={df_wx['weather_temp_c'].min():.1f}°C  "
443       f"temp_max={df_wx['weather_temp_c'].max():.1f}°C")
444
445
446 # ============================================================
447 # MERGE ALL FILES (NO1)
448 # ============================================================
449 print("\n" + "=" * 60)
450 print("SECTION 3 - MERGING ALL FILES (NO1)")
451 print("=" * 60)
452
453 # Zone-specific files for NO1
454 zone_files = [df_spot, df_epad_clean, df_load, df_wx]
455
456 # Merge zone-specific files together
457 df_zone = reduce(
458     lambda left, right: pd.merge(left, right, on='date', how='inner'),
459     zone_files
460 )
461
462 # Merge zone files with shared files
463 df_master = pd.merge(df_zone, df_shared, on='date', how='inner')
464 df_master = df_master.sort_values('date').reset_index(drop=True)
465
466 print(f"\n  Raw master shape  : {df_master.shape}")
467 print(f"  Date range        : {df_master['date'].min().date()} →
468 {df_master['date'].max().date()}")
469 print(f"  Nulls             : {df_master.isnull().sum().sum()}")
470
471 # Verify expected row count
472 assert len(df_master) == 2881, f"Expected 2881 rows, got {len(df_master)}"
473 print(f"  Row count check   : 2881")
474
475
476 # ============================================================
477 # FEATURE ENGINEERING
```

```
# ============================================================
print("\n" + "=" * 60)
print("SECTION 4 - FEATURE ENGINEERING")
print("=" * 60)

df = df_master.copy()
df = df.sort_values('date').reset_index(drop=True)


# - GROUP A: PRICE LAG AND ROLLING FEATURES ------------------
# shift(1) or more ensures no leakage - model never sees today's price
# min_periods = window size - only compute when full window is available

df['lag_price_1d']   = df['spot_price'].shift(1)    # yesterday
df['lag_price_7d']   = df['spot_price'].shift(7)    # 1 week ago
df['lag_price_30d']  = df['spot_price'].shift(30)   # 1 month ago
df['lag_price_365d'] = df['spot_price'].shift(365)  # same day last year

# Moving averages - shift(1) first so today's price is not included
df['ma_price_7d']  = df['spot_price'].shift(1).rolling(7,  min_periods=7).mean()
df['ma_price_30d'] = df['spot_price'].shift(1).rolling(30, min_periods=30).mean()
df['std_price_7d']  = df['spot_price'].shift(1).rolling(7,  min_periods=7).std()
df['std_price_30d'] = df['spot_price'].shift(1).rolling(30, min_periods=30).std()

print("  Group A - Price lag & rolling")


# - GROUP B: HYDRO DERIVED FEATURES ---------------------

# Week number (used for z-score calculation only - not a model feature)
df['week_num'] = df['date'].dt.isocalendar().week.astype(int)

# Inflow z-score: how abnormal is current inflow vs 68-year history?
# Positive = above norm (surplus water), Negative = below norm (drought →
higher prices)
# weekly_norm and weekly_std were computed from the full 1958-2026 NVE record
df['derived_inflow_z_score'] = (
    (df['inflow_hbv_gwh'] - df['week_num'].map(weekly_norm)) /
    df['week_num'].map(weekly_std)
)

# Daily reservoir change: positive = filling, negative = draining
df['derived_hydro_change'] = df['hydro_reserve_gwh'].diff(1)

print("  Group B - Hydro derived")


# - GROUP C: FORWARD CURVE DERIVED ----------------------
# Curve slope: Y1 - M1
# Positive (contango) = market expects prices to rise
# Negative (backwardation) = market expects prices to fall
df['derived_forward_slope'] = df['forward_Y1'] - df['forward_M1']

print("  Group C - Forward derived")

```

```
533
534 # - GROUP D: FUEL DERIVED --------------------------
535 # Gas price volatility: 30-day rolling standard deviation of TTF
536 # Captures uncertainty in marginal cost - important for probabilistic forecasting
537 # min_periods=30 so only valid when full 30-day window is available
538 df['derived_gas_volatility_30d'] = (
539     df['fuel_gas_eur_mwh'].rolling(30, min_periods=30).std()
540 )
541
542 print("  Group D - Fuel derived")
543
544
545 # - GROUP E: CALENDAR CYCLICAL ENCODING --------------------
546 # sin/cos encoding avoids the discontinuity at year-end (Dec → Jan would jump 12 →
547 1)
548 # 365.25 accounts for leap years
549 # cal_week_sin/cos excluded - r=0.9996 with cal_day_of_year(identical information)
550
551 df['cal_month_sin']       = np.sin(2 * np.pi * df['date'].dt.month / 12)
552 df['cal_month_cos']       = np.cos(2 * np.pi * df['date'].dt.month / 12)
553 df['cal_day_of_year_sin'] = np.sin(2 * np.pi * df['date'].dt.dayofyear / 365.25)
554 df['cal_day_of_year_cos'] = np.cos(2 * np.pi * df['date'].dt.dayofyear / 365.25)
555
556 print("  Group E - Calendar cyclical")
557
558
559 # ============================================================
560 # SELECT FINAL COLUMNS
561 # ============================================================
562 print("\n" + "=" * 60)
563 print("SECTION 5 - SELECTING FINAL 34 COLUMNS")
564 print("=" * 60)
565
566 # Columns REMOVED (with reason):
567 #   reservoir_deviation_gwh  → NVE official measure - KEPT (better than derived
568 alternative)
569 #   forward_M2/M3, Q1        → r > 0.89 with forward_M1 (redundant)
570 #   epad_Q1                  → r = 0.97 with epad_M1 (redundant)
571 #   weather_temp_max/min     → r > 0.95 with weather_temp_c (redundant)
572 #   cal_week_sin/cos         → r = 0.9996 with cal_day_of_year (identical)
573 #   no1_spot_std             → subtle leakage risk
574 #   week_num                 → internal variable only, not a model feature
575
576 final_cols = [
577     'date',
578     'spot_price',                   # TARGET - daily mean NO1 price (EUR/MWh)
579
580     # Raw hydro (4)
581     'hydro_reserve_gwh',            # national reservoir level (GWh)
582     'inflow_hbv_gwh',               # weekly water inflow (GWh)
583     'inflow_snow_gwh',              # snowpack energy (GWh)
584     'reservoir_deviation_gwh',      # NVE official deviation from long-run norm
585     (GWh)
586
587     # Raw forward (2)
```

```
    'forward_M1',                    # 1-month system forward price (EUR/MWh)
    'forward_Y1',                    # 1-year system forward price (EUR/MWh)

    # Raw EPAD (2)
    'epad_M1',                       # NO1 monthly area differential (EUR/MWh)
    'epad_Y1',                       # NO1 yearly area differential (EUR/MWh)

    # Raw fuel (3)
    'fuel_gas_eur_mwh',              # TTF gas price (EUR/MWh)
    'fuel_carbon_eur_t',             # EU carbon allowance (EUR/tonne CO2)
    'fuel_brent_usd_bbl',            # Brent crude oil (USD/barrel)

    # Raw weather (3) - max/min removed (r > 0.95 with temp_c)
    'weather_temp_c',                # daily average temperature (°C)
    'weather_wind_ms',               # daily wind speed (m/s)
    'weather_precip_mm',             # daily precipitation (mm)

    # Raw demand + macro (2)
    'load_mw',                       # NO1 electricity consumption (MW)
    'macro_eur_nok',                 # EUR/NOK exchange rate

    # Price lags (4)
    'lag_price_1d',
    'lag_price_7d',
    'lag_price_30d',
    'lag_price_365d',

    # Rolling statistics (4)
    'ma_price_7d',
    'ma_price_30d',
    'std_price_7d',
    'std_price_30d',

    # Hydro derived (2)
    'derived_inflow_z_score',
    'derived_hydro_change',

    # Forward derived (1)
    'derived_forward_slope',

    # Fuel derived (1)
    'derived_gas_volatility_30d',

    # Calendar (4)
    'cal_month_sin',
    'cal_month_cos',
    'cal_day_of_year_sin',
    'cal_day_of_year_cos',
]

# Verify all columns exist
missing = [c for c in final_cols if c not in df.columns]
assert not missing, f"Missing columns: {missing}"

df_final = df[final_cols].copy()
```

```
print(f"  Selected {df_final.shape[1]} columns  ({df_final.shape[1]-2} features +
date + target)")


# ============================================================
# CLEAN INF VALUES, PRESERVE LAG NaN WARMUP
# ============================================================
print("\n" + "=" * 60)
print("SECTION 6 - CLEANING AND PRESERVING WARMUP")
print("=" * 60)

# Lag columns must keep their NaN values - these are the warmup rows
# that will be removed by dropna() at the end
lag_cols = [
    'lag_price_1d', 'lag_price_7d', 'lag_price_30d', 'lag_price_365d',
    'ma_price_7d',  'ma_price_30d', 'std_price_7d',  'std_price_30d',
    'derived_hydro_change', 'derived_gas_volatility_30d'
]

# Non-lag columns: replace inf → ffill → bfill (safe to fill)
non_lag = [c for c in df_final.columns if c not in lag_cols and c != 'date']
df_final[non_lag]  = df_final[non_lag].replace([np.inf, -np.inf], np.nan).ffill()
.bfill()

# Lag columns: replace inf → NaN only (preserve NaN warmup)
df_final[lag_cols] = df_final[lag_cols].replace([np.inf, -np.inf], np.nan)


# ============================================================
# ASSERT CHECKS AND SAVE
# ============================================================
print("\n" + "=" * 60)
print("SECTION 7 - FINAL CHECKS AND SAVING")
print("=" * 60)

# Checks on full dataset (with warmup)
assert len(df_final)    == 2881,   f"Expected 2881 rows, got {len(df_final)}"
assert df_final.shape[1] == 34,    f"Expected 34 columns, got {df_final.shape[1]}"
assert np.isinf(df_final.select_dtypes(include=np.number)).sum().sum() == 0,
"Inf found!"
assert pd.isna(df_final.loc[0,   'lag_price_365d']), "lag NaN not preserved on
row 0!"
assert pd.notna(df_final.loc[365, 'lag_price_365d']), "lag should be valid on
row 365!"

# Save master with warmup (reference file)
df_final.to_csv(DATA_FOLDER / 'master_dataset_NO1.csv', index=False)

# - DROP WARMUP AND SAVE CLEAN DATASET --------------------
df_clean = df_final.dropna().reset_index(drop=True)

# Checks on clean dataset
assert len(df_clean)    == 2516,   f"Expected 2516 rows, got {len(df_clean)}"
assert df_clean.shape[1] == 34,    f"Expected 34 columns, got {df_clean.shape[1]}"
assert df_clean.isnull().sum().sum()  == 0,  "Nulls in clean dataset!"
```

```
assert np.isinf(df_clean.select_dtypes(include=np.number)).sum().sum() == 0,
"Inf in clean!"
assert df_clean.duplicated(subset=['date']).sum() == 0, "Duplicate dates!"
assert df_clean['date'].min() == pd.Timestamp('2018-11-24'), "Wrong start date!"
assert df_clean['date'].max() == pd.Timestamp('2025-10-13'), "Wrong end date!"

df_clean.to_csv(DATA_FOLDER / 'master_clean_NO1.csv', index=False)

print(f"\n  master_dataset_NO1.csv  : {df_final.shape}  (includes NaN warmup
rows)")
print(f"  master_clean_NO1.csv    : {df_clean.shape}   (model ready - warmup
dropped)")
print(f"\n  Date range (clean)      : {df_clean['date'].min().date()} →
{df_clean['date'].max().date()}")
print(f"  Target mean             : {df_clean['spot_price'].mean():.2f} EUR/MWh")
print(f"  Target std              : {df_clean['spot_price'].std():.2f} EUR/MWh")
print(f"  Target min              : {df_clean['spot_price'].min():.2f} EUR/MWh")
print(f"  Target max              : {df_clean['spot_price'].max():.2f} EUR/MWh")
print(f"  Negative price days     : {(df_clean['spot_price'] < 0).sum()}")
print(f"\n  All assert checks passed")
print(f"\n{'='*60}")
```

Appendix C

Source Code for Exploratory Data Analysis (EDA)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.patches import Patch
from scipy.stats import pearsonr, spearmanr, johnsonsu
from statsmodels.tsa.stattools import acf, pacf, adfuller

plt.rcParams.update({
    'figure.facecolor': 'white',  'axes.facecolor': '#f8f9fa',
    'axes.grid': True,            'grid.alpha': 0.35,
    'font.size': 11,              'axes.spines.top': False,
    'axes.spines.right': False,   'axes.labelsize': 12,
    'axes.titlesize': 13,         'figure.dpi': 110,
    'savefig.dpi': 150,           'savefig.bbox': 'tight',
})

COLORS = {
    'pre':    '#2196F3',   # blue  - pre-crisis
    'crisis': '#F44336',   # red   - crisis 2021-22
    'post':   '#4CAF50',   # green - post-crisis
    'hydro':  '#00BCD4',   'gas':  '#FF9800',   'fwd': '#9C27B0',
}
CRISIS_START = pd.Timestamp('2021-07-01')
CRISIS_END   = pd.Timestamp('2023-01-01')
MONTHS = ['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec']

DATA_PATH = 'master_clean_NO1.csv'

df = pd.read_csv(DATA_PATH, parse_dates=['date'])
df = df.sort_values('date').reset_index(drop=True)

print(f"Rows    : {df.shape[0]}")
print(f"Columns : {df.shape[1]}")
print(f"From    : {df['date'].min().date()}")
print(f"To      : {df['date'].max().date()}")
print(f"Missing : {df.isnull().sum().sum()} values  (0 = clean)")
print()

```

```
# Feature groups
groups = {
    'Target':          ['spot_price'],
    'Hydro':           ['hydro_reserve_gwh','inflow_hbv_gwh','inflow_snow_gwh',
                        'reservoir_deviation_gwh','derived_inflow_z_score',
                        'derived_hydro_change'],
    'Forwards/EPAD':   ['forward_M1','forward_Y1','epad_M1','epad_Y1',
                        'derived_forward_slope'],
    'Fuel Prices':     ['fuel_gas_eur_mwh','fuel_carbon_eur_t','fuel_brent_usd_bbl',
                        'derived_gas_volatility_30d'],
    'Weather':         ['weather_temp_c','weather_wind_ms','weather_precip_mm'],
    'Load/Macro':      ['load_mw','macro_eur_nok'],
    'AR Lags':         ['lag_price_1d','lag_price_7d','lag_price_30d',
    'lag_price_365d', 'ma_price_7d', 'ma_price_30d', 'std_price_7d',
    'Calendar':        ['cal_month_sin','cal_month_cos','cal_day_of_year_sin',
                        'cal_day_of_year_cos'],
}
print("Feature groups:")
print("-"*40)
for name, cols in groups.items():
    n = len([c for c in cols if c in df.columns])
    print(f"  {name:<18}  {n} columns")
print("-"*40)
print(f"  {'TOTAL (features)':<18}  {df.shape[1]-2}")
def get_regime(d):
    if d < CRISIS_START:   return 'Pre-Crisis (2018-mid2021)'
    elif d < CRISIS_END:   return 'Crisis (mid2021-2022)'
    return 'Post-Crisis (2023-2025)'

df['regime'] = df['date'].apply(get_regime)
df['year']   = df['date'].dt.year
df['month']  = df['date'].dt.month

REGIME_COLORS = {
    'Pre-Crisis (2018-mid2021)': COLORS['pre'],
    'Crisis (mid2021-2022)':     COLORS['crisis'],
    'Post-Crisis (2023-2025)':   COLORS['post'],
}

# Annual statistics
annual = df.groupby('year')['spot_price'].agg(
    Mean='mean', Std='std', Min='min', Max='max',
    Neg_days=lambda x: (x < 0).sum()
).round(2)
print("Table 1 - Annual Spot Price Statistics (EUR/MWh):")
print("="*60)
print(annual.to_string())
print("="*60)
print()

fig, axes = plt.subplots(3, 1, figsize=(15, 12), gridspec_kw={'hspace': 0.45})
fig.suptitle('Figure 1 - NO1 Spot Price Dynamics and Regime Analysis',
             fontsize=15, fontweight='bold', y=0.98)

# Panel A: daily prices coloured by regime
```

```
ax = axes[0]
for regime, grp in df.groupby('regime', sort=False):
    ax.plot(grp['date'], grp['spot_price'],
            color=REGIME_COLORS[regime], lw=0.75, label=regime)
ax.axvspan(CRISIS_START, CRISIS_END, alpha=0.08, color='red',
label='Crisis Window')
mean_p = df['spot_price'].mean()
ax.axhline(mean_p, color='k', ls='--', lw=1.0,
           label=f'Full-sample mean = {mean_p:.1f} EUR/MWh')
ax.set_title('Panel A - Daily Spot Price (Three Distinct Regimes)')
ax.set_ylabel('EUR/MWh')
ax.legend(fontsize=9, ncol=3)

# Panel B: monthly mean ± 1 std
ax = axes[1]
mon = df.resample('ME', on='date')['spot_price'].agg(['mean', 'std'])
.reset_index()
ax.fill_between(mon['date'], mon['mean'] - mon['std'], mon['mean'] + mon['std'],
                alpha=0.25, color='steelblue', label='±1 Std Dev')
ax.plot(mon['date'], mon['mean'], color='steelblue', lw=2, label='Monthly Mean')
ax.axvspan(CRISIS_START, CRISIS_END, alpha=0.08, color='red')
ax.set_title('Panel B - Monthly Average Price ± 1 Standard Deviation')
ax.set_ylabel('EUR/MWh')
ax.legend(fontsize=9)

# Panel C: annual box plots
ax = axes[2]
years = sorted(df['year'].unique())
data_by_year = [df[df['year'] == y]['spot_price'].values for y in years]
bp = ax.boxplot(data_by_year, patch_artist=True,
                medianprops=dict(color='k', lw=2),
                flierprops=dict(marker='.', ms=3, alpha=0.3))
for patch, yr in zip(bp['boxes'], years):
    if yr in {2021, 2022}: c = COLORS['crisis']
    elif yr < 2021:        c = COLORS['pre']
    else:                  c = COLORS['post']
    patch.set_facecolor(c)
    patch.set_alpha(0.75)
ax.set_xticklabels(years)
ax.set_title('Panel C - Annual Distribution (Crisis Years in Red)')
ax.set_ylabel('EUR/MWh')
ax.set_xlabel('Year')
ax.legend(handles=[
    Patch(fc=COLORS['pre'],    alpha=0.75, label='Pre-Crisis'),
    Patch(fc=COLORS['crisis'], alpha=0.75, label='Crisis 2021-22'),
    Patch(fc=COLORS['post'],   alpha=0.75, label='Post-Crisis'),
], fontsize=9)
plt.show()

# Hydro Fundamentals (Norway-Specific)
fig = plt.figure(figsize=(16, 13))
fig.suptitle('Figure 3 - Hydro Fundamentals - The Norwegian Price Driver',
             fontsize=14, fontweight='bold', y=0.98)
gs = gridspec.GridSpec(3, 2, figure=fig, hspace=0.52, wspace=0.35)

```

```
# Panel A: reservoir level vs spot price (dual axis, full timeline)
ax = fig.add_subplot(gs[0, :])
ax_r = ax.twinx()
ax.fill_between(df['date'], df['hydro_reserve_gwh'] / 1000,
                alpha=0.35, color=COLORS['hydro'], label='Hydro Reserve (TWh)')
ax.plot(df['date'], df['hydro_reserve_gwh'] / 1000, color=COLORS['hydro'], lw=0.8)
ax_r.plot(df['date'], df['spot_price'], color=COLORS['crisis'],
          lw=0.9, alpha=0.85, label='Spot Price')
ax.set_ylabel('Hydro Reserve (TWh)', color=COLORS['hydro'])
ax_r.set_ylabel('Spot Price (EUR/MWh)', color=COLORS['crisis'])
ax.tick_params(axis='y', colors=COLORS['hydro'])
ax_r.tick_params(axis='y', colors=COLORS['crisis'])
l1, n1 = ax.get_legend_handles_labels()
l2, n2 = ax_r.get_legend_handles_labels()
ax.legend(l1 + l2, n1 + n2, fontsize=9, loc='upper left')
ax.set_title('Panel A - Hydro Reservoir Level vs Spot Price
(Inverse Relationship)')

# Panel B: reservoir deviation vs spot price scatter (coloured by month)
ax2 = fig.add_subplot(gs[1, 0])
sc = ax2.scatter(df['reservoir_deviation_gwh'] / 1000, df['spot_price'],
                 c=df['month'], cmap='hsv', alpha=0.35, s=8)
plt.colorbar(sc, ax=ax2, label='Month')
rho_dev, _ = spearmanr(df['reservoir_deviation_gwh'], df['spot_price'])
z = np.polyfit(df['reservoir_deviation_gwh'] / 1000, df['spot_price'], 1)
xl = np.linspace(df['reservoir_deviation_gwh'].min() / 1000,
                 df['reservoir_deviation_gwh'].max() / 1000, 100)
ax2.plot(xl, np.poly1d(z)(xl), 'k--', lw=1.5, label='Linear fit')
ax2.axvline(0, color='gray', ls=':', lw=1, label='Seasonal normal')
ax2.set_title(f'Panel B - Reservoir Deviation vs Price (Spearman rho =
{rho_dev:.3f})')
ax2.set_xlabel('Deviation from Seasonal Normal (TWh)')
ax2.set_ylabel('Spot Price (EUR/MWh)')
ax2.legend(fontsize=8)

# Panel C: seasonal hydro patterns by month (dual axis)
ax3 = fig.add_subplot(gs[1, 1])
mh  = df.groupby('month')['hydro_reserve_gwh'].mean() / 1000
mi  = df.groupby('month')['inflow_hbv_gwh'].mean()
ax3r = ax3.twinx()
ax3.bar(range(1, 13), mh.values, alpha=0.6, color=COLORS['hydro'],
        label='Avg Reserve (TWh)')
ax3r.plot(range(1, 13), mi.values, 'o-', color=COLORS['gas'],
          lw=2, label='Avg Inflow (GWh/wk)')
ax3.set_xticks(range(1, 13))
ax3.set_xticklabels([m[0] for m in zip(MONTHS)], fontsize=8)
ax3.set_ylabel('Reservoir (TWh)', color=COLORS['hydro'])
ax3r.set_ylabel('Inflow (GWh/week)', color=COLORS['gas'])
ax3.tick_params(axis='y', colors=COLORS['hydro'])
ax3r.tick_params(axis='y', colors=COLORS['gas'])
l1, n1 = ax3.get_legend_handles_labels()
l2, n2 = ax3r.get_legend_handles_labels()
ax3.legend(l1 + l2, n1 + n2, fontsize=8)
ax3.set_title('Panel C - Seasonal Hydro Patterns (snowmelt peak May-Jun)')

```

```
#Panel D: hydro statistics by regime (table)
ax4 = fig.add_subplot(gs[2, 0])
ax4.axis('off')
rows = []
rc   = ['#BBDEFB', '#FFCDD2', '#C8E6C9']
for regime, grp in df.groupby('regime', sort=False):
    rh, _ = spearmanr(grp['hydro_reserve_gwh'], grp['spot_price'])
    rows.append([
        regime.split('(')[0].strip(),
        f"{grp['hydro_reserve_gwh'].mean() / 1000:.1f} TWh",
        f"{grp['inflow_hbv_gwh'].mean():.0f} GWh/wk",
        f"{rh:.3f}",
    ])
tbl = ax4.table(
    cellText=rows,
    colLabels=['Regime', 'Avg Reserve', 'Avg Inflow', 'Hydro-Price rho'],
    loc='center', cellLoc='center',
)
tbl.auto_set_font_size(False)
tbl.set_fontsize(9)
tbl.scale(1, 2.0)
for (r, c), cell in tbl.get_celld().items():
    if r == 0:
        cell.set_facecolor('#1565C0')
        cell.set_text_props(color='white', fontweight='bold')
    elif r <= 3:
        cell.set_facecolor(rc[r - 1])
ax4.set_title('Panel D - Hydro Stats by Regime', fontsize=10, pad=8)
plt.show()

# Time-Series Properties, ACF and  Unit Root Tests - Non-Stationarity
price = df['spot_price'].values

fig = plt.figure(figsize=(16, 12))
fig.suptitle('Time-Series Properties: ACF, Distribution, Unit Root Tests',
             fontsize=14, fontweight='bold', y=0.98)
gs = gridspec.GridSpec(3, 2, figure=fig, hspace=0.55, wspace=0.35)


#ACF at key forecasting horizons (bar chart - key for model design)
ax4 = fig.add_subplot(gs[1, 1])
acf_full     = acf(price, nlags=365)
h_labels     = [1, 7, 14, 30, 60, 90, 120, 180, 270, 365]
acf_at_h     = [float(acf_full[min(h, len(acf_full) - 1)]) for h in h_labels]
bar_colors   = ['#4CAF50' if v > 0.3 else '#FFC107' if v > 0.1 else '#F44336'
                for v in acf_at_h]
bars = ax4.bar(range(len(h_labels)), acf_at_h, color=bar_colors, alpha=0.85)
ax4.set_xticks(range(len(h_labels)))
ax4.set_xticklabels([f'{h}d' for h in h_labels], fontsize=9)
ax4.axhline(0.3, color='green',  ls='--', lw=1.2, label='0.3 = useful')
ax4.axhline(0.1, color='orange', ls='--', lw=1.2, label='0.1 = marginal')
ax4.axhline(0,   color='k', lw=0.8)
for bar, val in zip(bars, acf_at_h):
    ax4.text(bar.get_x() + bar.get_width() / 2,
             bar.get_height() + 0.01, f'{val:.3f}',
```

```
            ha='center', va='bottom', fontsize=8)
ax4.set_title('Panel D - ACF at Key Horizons\n'
              'Green = useful, Orange = marginal, Red = spurious risk')
ax4.set_ylabel('Autocorrelation')
ax4.legend(fontsize=9)

# ADF unit root test table
ax5 = fig.add_subplot(gs[2, :])
ax5.axis('off')
adf_rows = []
series_to_test = [
    ('Spot Price',          df['spot_price']),
    ('Gas Price',           df['fuel_gas_eur_mwh']),
    ('CO2 Price',           df['fuel_carbon_eur_t']),
    ('Forward M1',          df['forward_M1']),
    ('Forward Y1',          df['forward_Y1']),
    ('Hydro Reserve',       df['hydro_reserve_gwh']),
    ('Reservoir Deviation', df['reservoir_deviation_gwh']),
    ('EUR/NOK',             df['macro_eur_nok']),
]
for name, series in series_to_test:
    try:
        stat, p, _, _, crit, _ = adfuller(series.dropna(), maxlag=10)
        result = 'UNIT ROOT' if p > 0.05 else 'Stationary'
        impl   = ('Spurious regression risk at long horizons'
                  if p > 0.05 else 'Safe to use as regressor')
        adf_rows.append([name, f'{stat:.3f}', f'{p:.4f}',
                         f'{crit["1%"]:.3f}', result, impl])
    except Exception as e:
        adf_rows.append([name, 'N/A', 'N/A', 'N/A', 'N/A', str(e)])

tbl = ax5.table(
    cellText=adf_rows,
    colLabels=['Series', 'ADF Stat', 'p-value', 'Crit(1%)',
               'Conclusion', 'Implication for Long-Horizon Models'],
    loc='center', cellLoc='center',
)
tbl.auto_set_font_size(False)
tbl.set_fontsize(9)
tbl.scale(1, 1.65)
for (r, c), cell in tbl.get_celld().items():
    if r == 0:
        cell.set_facecolor('#1565C0')
        cell.set_text_props(color='white', fontweight='bold')
    elif 'UNIT ROOT' in str(cell.get_text().get_text()):
        cell.set_facecolor('#FFCDD2')
    elif 'Stationary' in str(cell.get_text().get_text()):
        cell.set_facecolor('#C8E6C9')
    elif r % 2 == 0:
        cell.set_facecolor('#f5f5f5')
ax5.set_title('Panel E - ADF Unit Root Tests (H0: series has a unit root)\n'
              'Critical finding: spot, gas, CO2, forwards all have unit roots',
              fontsize=11, pad=8)
plt.show()
```

```
print()
print("ACF values at key horizons:")
print("-"*55)
for h, v in zip(h_labels, acf_at_h):
    if v > 0.3:   flag = "USEFUL - include AR lags"
    elif v > 0.1: flag = "marginal"
    else:         flag = "near-zero - spurious regression risk"
    print(f"  h = {h:3d}d:  ACF = {v:.4f}   {flag}")
print()

# Feature Relevance by Forecasting Horizon
fig, axes = plt.subplots(1, 2, figsize=(16, 7))
fig.suptitle('Feature Relevance Decay Across Forecasting Horizons\n',
             fontsize=13, fontweight='bold')

horizons = [1, 7, 14, 30, 60, 90, 120, 180, 270, 360]

# Panel A: fundamental features
ax = axes[0]
fundamentals = [
    ('Gas Price',           'fuel_gas_eur_mwh',        COLORS['gas']),
    ('Forward M+1',         'forward_M1',               COLORS['pre']),
    ('Forward Y+1',         'forward_Y1',               COLORS['fwd']),
    ('Hydro Reserve',       'hydro_reserve_gwh',        COLORS['hydro']),
    ('Reservoir Deviation', 'reservoir_deviation_gwh',  '#009688'),
    ('CO2 Price',           'fuel_carbon_eur_t',        '#E91E63'),
    ('EUR/NOK',             'macro_eur_nok',            '#795548'),
]
for label, col, color in fundamentals:
    corrs = []
    for h in horizons:
        if h >= len(df):
            corrs.append(np.nan)
            continue
        shifted = df['spot_price'].shift(-h)
        valid   = ~(shifted.isna() | df[col].isna())
        r, _    = pearsonr(df.loc[valid, col], shifted[valid])
        corrs.append(abs(r))
    ax.plot(horizons, corrs, 'o-', lw=1.8, ms=5, label=label, color=color)

ax.axvline(30,  color='gray', ls=':', lw=1.2)
ax.axvline(180, color='gray', ls=':', lw=1.2)
ax.text(15,  0.02, '1mo', fontsize=8, ha='center', color='gray')
ax.text(150, 0.02, '6mo', fontsize=8, ha='center', color='gray')
ax.axhline(0.3, color='green',  ls='--', lw=1, alpha=0.7)
ax.axhline(0.1, color='orange', ls='--', lw=1, alpha=0.7)
ax.set_title('Panel A - Fundamental Features\n(Stay informative at
medium/long horizons)')
ax.set_xlabel('Forecasting Horizon (days)')
ax.set_ylabel('|Pearson r| with future spot price')
ax.legend(fontsize=8, ncol=2)
ax.set_ylim(0, 1)
ax.set_xticks(horizons)

# Panel B: AR features - showing rapid decay
```

```
ax2 = axes[1]
ar_features = [
    ('ACF: Spot Price (auto)', 'spot_price',    'steelblue'),
    ('Lag 1d',                 'lag_price_1d',  COLORS['crisis']),
    ('Lag 7d',                 'lag_price_7d',  COLORS['gas']),
    ('Lag 30d',                'lag_price_30d', COLORS['pre']),
    ('MA 7d',                  'ma_price_7d',   '#9C27B0'),
    ('MA 30d',                 'ma_price_30d',  '#607D8B'),
]
for label, col, color in ar_features:
    corrs = []
    for h in horizons:
        if h >= len(df):
            corrs.append(np.nan)
            continue
        if col == 'spot_price':
            corrs.append(float(acf_full[min(h, len(acf_full) - 1)]))
        else:
            shifted = df['spot_price'].shift(-h)
            valid   = ~(shifted.isna() | df[col].isna())
            r, _    = pearsonr(df.loc[valid, col], shifted[valid])
            corrs.append(abs(r))
    ax2.plot(horizons, corrs, 'o-', lw=1.8, ms=5, label=label, color=color)

ax2.axvline(30,  color='gray', ls=':', lw=1.2)
ax2.axvline(180, color='gray', ls=':', lw=1.2)
ax2.axhline(0.3, color='green',  ls='--', lw=1, alpha=0.7, label='0.3 threshold')
ax2.axhline(0.1, color='orange', ls='--', lw=1, alpha=0.7, label='0.1 threshold')
ax2.set_title('Panel B - AR/Lag Features: Rapid Decay\n'
ax2.set_xlabel('Forecasting Horizon (days)')
ax2.set_ylabel('|Correlation| with future spot')
ax2.legend(fontsize=8, ncol=2)
ax2.set_ylim(0, 1)
ax2.set_xticks(horizons)

plt.tight_layout()
plt.show()
```

Appendix D

Source Code for Feature Engineering Results

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.patches import Patch
from scipy.stats import pearsonr, spearmanr

plt.rcParams.update({
    'figure.facecolor': 'white',  'axes.facecolor': '#f8f9fa',
    'axes.grid': True,            'grid.alpha': 0.35,
    'font.size': 11,              'axes.spines.top': False,
    'axes.spines.right': False,   'axes.labelsize': 12,
    'axes.titlesize': 13,         'figure.dpi': 110,
    'savefig.dpi': 150,           'savefig.bbox': 'tight',
})

COLORS = {
    'pre':    '#2196F3', 'crisis': '#F44336', 'post': '#4CAF50',
    'hydro':  '#00BCD4', 'gas':    '#FF9800', 'fwd':  '#9C27B0',
    'short':  '#1976D2', 'medium': '#F57C00', 'long': '#388E3C',
}
CRISIS_START = pd.Timestamp('2021-07-01')
CRISIS_END   = pd.Timestamp('2023-01-01')
TRAIN_END    = pd.Timestamp('2021-12-31')   # end of training window
VAL_START    = pd.Timestamp('2022-01-01')   # validation = crisis stress-test
VAL_END      = pd.Timestamp('2022-12-31')
TEST_START   = pd.Timestamp('2023-01-01')
TEST_END     = pd.Timestamp('2024-12-31')

MONTHS = ['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec']

DATA_PATH = 'master_clean_NO1.csv'

df = pd.read_csv(DATA_PATH, parse_dates=['date'])
df = df.sort_values('date').reset_index(drop=True)

# Add regime and split labels
def get_regime(d):
    if d < CRISIS_START:   return 'Pre-Crisis (2018-mid2021)'
```

```
    elif d < CRISIS_END:   return 'Crisis (mid2021-2022)'
    return 'Post-Crisis (2023-2025)'

def get_split(d):
    if d <= TRAIN_END:                          return 'train'
    elif VAL_START <= d <= VAL_END:             return 'val'
    elif TEST_START <= d <= TEST_END:           return 'test'
    return 'holdout'

df['regime'] = df['date'].apply(get_regime)
df['split']  = df['date'].apply(get_split)
df['year']   = df['date'].dt.year
df['month']  = df['date'].dt.month
df['dow']    = df['date'].dt.dayofweek
df['iso_week'] = df['date'].dt.isocalendar().week.astype(int)

# Split counts
split_counts = df['split'].value_counts()
print("Train / Validation / Test / Holdout Split:")
print("="*50)
for split_name, dates in [
    ('TRAIN',   (df['date'].min(), TRAIN_END)),
    ('VAL',     (VAL_START, VAL_END)),
    ('TEST',    (TEST_START, TEST_END)),
    ('HOLDOUT', (df['date'][df['split']=='holdout'].min() if
    (df['split']=='holdout').any() else None,
                 df['date'].max())),
]:
    n = (df['split'] == split_name.lower()).sum()
    if dates[0] is not None:
        print(f"  {split_name:<8}: {str(dates[0])[:10]} to {str(dates[1])[:10]}
        ({n} days)")
print("="*50)
print()
print("Role of each split:")
print("  TRAIN   : Model learns relationships from pre-crisis period")
print("  VAL     : 2022 crisis - used as STRESS TEST, not for tuning")
print("  TEST    : Post-crisis period - primary evaluation set")
print("  HOLDOUT : 2025 data - final out-of-sample evaluation")

def build_seasonal_profile(df, col, train_end=TRAIN_END):
    """
    Compute seasonal median and std by (month, iso_week)
    using ONLY training data (data before train_end).

    This is the Norwegian analog of the GAM-based RES/load forecasts
    in Ghelasi & Ziel (2025) - we use the seasonal mean as a proxy
    for the expected value at medium/long horizons, avoiding lookahead bias.

    Parameters
    ----------
    df       : DataFrame with 'date', 'month', 'iso_week', and col columns
    col      : Column name to compute seasonal profile for
    train_end: Only use data up to this date (no lookahead)

```

```
    Returns
    -------
    DataFrame with columns: month, iso_week, seasonal_mean, seasonal_std,
                            seasonal_median, seasonal_p10, seasonal_p90
    """
    train = df[df['date'] <= train_end].copy()
    profile = (
        train.groupby(['month', 'iso_week'])[col]
        .agg(
            seasonal_mean   = 'mean',
            seasonal_median = 'median',
            seasonal_std    = 'std',
            seasonal_p10    = lambda x: np.percentile(x, 10),
            seasonal_p90    = lambda x: np.percentile(x, 90),
            n_obs           = 'count',
        )
        .reset_index()
    )
    # Smooth with 3-period rolling window to reduce noise
    for col_s in ['seasonal_mean', 'seasonal_median', 'seasonal_std']:
        profile[col_s] = profile[col_s].rolling(3, center=True, min_periods=1)
        .mean()
    return profile


# Build seasonal profiles for key variables
print("Building seasonal profiles from TRAINING DATA ONLY")
print(f"(using data up to {TRAIN_END.date()} to avoid lookahead bias)")
print()

cols_to_profile = {
    'spot_price':             'Spot Price (EUR/MWh)',
    'hydro_reserve_gwh':      'Hydro Reserve (GWh)',
    'inflow_hbv_gwh':         'Inflow HBV (GWh/week)',
    'reservoir_deviation_gwh':'Reservoir Deviation (GWh)',
    'fuel_gas_eur_mwh':       'Gas Price (EUR/MWh)',
    'load_mw':                'Load (MW)',
}

seasonal_profiles = {}
for col, label in cols_to_profile.items():
    if col not in df.columns:
        print(f"  SKIP: {col} not in dataset")
        continue
    profile = build_seasonal_profile(df, col)
    seasonal_profiles[col] = profile
    n_weeks = len(profile)
    print(f"  {label:<30}  {n_weeks} (month, week) combinations built")

print()
print(f"Total profiles built: {len(seasonal_profiles)}")
print()
print("These profiles will be used as feature values at medium/long horizons")

def attach_seasonal_features(df, seasonal_profiles):
```

```
    """
    Join seasonal profile statistics onto the full DataFrame.
    Adds columns: seasonal_{col}_mean, seasonal_{col}_std,
                  seasonal_{col}_p10, seasonal_{col}_p90
    for each variable in seasonal_profiles.

    IMPORTANT: These seasonal values are computed from training data only
    (no lookahead bias). They can be used as regressors at ALL horizons
    because they represent 'what is the typical value at this time of year'
    - information that is always available at forecast time.
    """
    df = df.copy()
    for col, profile in seasonal_profiles.items():
        merged = df[['month', 'iso_week']].merge(
            profile[['month', 'iso_week',
                     'seasonal_mean', 'seasonal_std',
                     'seasonal_p10', 'seasonal_p90']],
            on=['month', 'iso_week'],
            how='left',
        )
        df[f'seasonal_{col}_mean'] = merged['seasonal_mean'].values
        df[f'seasonal_{col}_std']  = merged['seasonal_std'].values
        df[f'seasonal_{col}_p10']  = merged['seasonal_p10'].values
        df[f'seasonal_{col}_p90']  = merged['seasonal_p90'].values
    return df

df = attach_seasonal_features(df, seasonal_profiles)

# Show what was added
new_cols = [c for c in df.columns if c.startswith('seasonal_')]
print(f"Added {len(new_cols)} seasonal feature columns:")
for c in new_cols:
    non_null = df[c].notna().sum()
    print(f"  {c:<45}  {non_null} non-null values")
print()
print("Key derived features from seasonal profiles:")
print()
# Price vs seasonal mean (the 'current model' deviation signal)
df['price_vs_seasonal']  = df['spot_price'] - df['seasonal_spot_price_mean']
df['price_seasonal_z']   = df['price_vs_seasonal'] /
df['seasonal_spot_price_std'].clip(lower=1)
df['hydro_seasonal_z']   = ((df['hydro_reserve_gwh'] -
df['seasonal_hydro_reserve_gwh_mean'])
                            / df['seasonal_hydro_reserve_gwh_std'].clip(lower=1))
df['inflow_seasonal_z']  = ((df['inflow_hbv_gwh'] -
df['seasonal_inflow_hbv_gwh_mean'])
                            / df['seasonal_inflow_hbv_gwh_std'].clip(lower=1))

print("  price_vs_seasonal : current spot minus seasonal expectation")
print(f"range: {df['price_vs_seasonal'].min():.1f} to {df['price_vs_seasonal']
.max():.1f} EUR/MWh")
print()
print("  price_seasonal_z  : standardised deviation from seasonal mean")
print(f"range: {df['price_seasonal_z'].min():.2f} to {df['price_seasonal_z']
.max():.2f} sigma")
```

```
print()
print("  hydro_seasonal_z  : how far current reservoir is from seasonal normal")
print(f"range: {df['hydro_seasonal_z'].min():.2f} to {df['hydro_seasonal_z']
.max():.2f} sigma")

# Engineer Short-Term Features
def engineer_short_term(df):
    """
    SHORT-TERM feature engineering (h <= 30 days).

    At short horizons ALL features are valid:
      - Autoregressive lags (ACF still > 0.6 at h=30d)
      - Forward prices (M1, Y1)
      - Actual hydro levels (known at forecast time via data feeds)
      - Weather forecasts (1-day-ahead available from met services)
      - Fuel prices (current front-month futures)

    Additional engineered features:
      - Rolling price statistics at multiple windows
      - Hydro-gas interaction (key Norwegian non-linearity)
      - Forward term structure (contango/backwardation signal)
      - Price momentum (deviation from recent average)
    """
    df = df.copy()

    # - Rolling price statistics (shift(1) to avoid lookahead) ------
    for w in [7, 14, 30, 90]:
        df[f'roll_price_mean_{w}d'] = df['spot_price'].shift(1).rolling(w).mean()
        df[f'roll_price_std_{w}d']  = df['spot_price'].shift(1).rolling(w).std()

    # - Rolling hydro and gas stats --------------------
    for w in [7, 30]:
        df[f'roll_hydro_mean_{w}d'] = df['hydro_reserve_gwh'].shift(1)
        .rolling(w).mean()
        df[f'roll_gas_mean_{w}d']   = df['fuel_gas_eur_mwh'].shift(1)
        .rolling(w).mean()

    # - Price momentum (current price vs rolling mean) ----------
    for w in [7, 30]:
        df[f'price_momentum_{w}d'] = df['lag_price_1d'] -
        df[f'roll_price_mean_{w}d']

    # - Hydro-gas interaction (key Norwegian non-linearity) --------
    # When hydro is LOW and gas is HIGH => extreme price spikes
    # Capture this interaction as a product of Z-scores
    hydro_z = ((df['hydro_reserve_gwh'] - df['hydro_reserve_gwh'].mean())
               / df['hydro_reserve_gwh'].std())
    gas_z   = ((df['fuel_gas_eur_mwh'] - df['fuel_gas_eur_mwh'].mean())
               / df['fuel_gas_eur_mwh'].std())
    df['interaction_hydro_gas'] = hydro_z * gas_z
    df['ratio_gas_to_hydro']    = (df['fuel_gas_eur_mwh']
                                   / (df['hydro_reserve_gwh'].clip(lower=1000)
                                   / 1000))

    # - Reservoir fill rate ------------------------
```

```
    df['hydro_fill_rate'] = (df['hydro_reserve_gwh'].diff()
                             / df['inflow_hbv_gwh'].clip(lower=1))
    df['hydro_fill_rate'] = df['hydro_fill_rate'].clip(-5, 5)

    # - Snow fraction (snow inflow / total inflow) ------------
    df['snow_fraction'] = df['inflow_snow_gwh'] / df['inflow_hbv_gwh']
    .clip(lower=1)

    # - Forward market features ---------------------
    df['epad_slope']      = df['epad_M1'] - df['epad_Y1']   # EPAD term structure
    df['implied_spot_M1'] = df['forward_M1'] + df['epad_M1']  # implied NO1 price
    df['implied_spot_Y1'] = df['forward_Y1'] + df['epad_Y1']
    df['spot_fwd_basis']  = df['spot_price'] - df['implied_spot_M1']
    df['gas_co2_ratio']   = df['fuel_gas_eur_mwh'] / df['fuel_carbon_eur_t']
    .clip(lower=1)

    # - Calendar dummies ------------------------
    m = df['date'].dt.month
    df['season_winter'] = ((m <= 2) | (m == 12)).astype(int)
    df['season_spring'] = ((m >= 3) & (m <= 5)).astype(int)
    df['season_summer'] = ((m >= 6) & (m <= 8)).astype(int)
    df['season_autumn'] = ((m >= 9) & (m <= 11)).astype(int)
    df['is_weekend']    = (df['date'].dt.dayofweek >= 5).astype(int)
    df['month_num']     = df['date'].dt.month
    df['week_of_year']  = df['date'].dt.isocalendar().week.astype(int)

    return df

df = engineer_short_term(df)

new_feats = [
    'roll_price_mean_7d', 'roll_price_std_7d', 'roll_price_mean_30d',
    'roll_hydro_mean_30d', 'roll_gas_mean_30d',
    'price_momentum_7d', 'price_momentum_30d',
    'interaction_hydro_gas', 'ratio_gas_to_hydro',
    'hydro_fill_rate', 'snow_fraction',
    'epad_slope', 'implied_spot_M1', 'implied_spot_Y1', 'spot_fwd_basis',
    'gas_co2_ratio', 'season_winter', 'is_weekend',
]
print("Short-term engineered features added:")
for f in new_feats:
    if f in df.columns:
        print(f"  {f:<30}  min={df[f].min():.2f}  max={df[f].max():.2f}")
print()
print(f"Total columns now: {df.shape[1]}")

# Define Horizon-Specific Feature Sets
# ============================================================================
# HORIZON-SPECIFIC FEATURE SETS
# Based on EDA findings + theoretical framework from:
#   - Ghelasi & Ziel (2025): AR terms and fuel lags become spurious beyond 30d
#   - Agakishiev et al. (2025): Forward prices and hydro are medium-term anchors
# ============================================================================

FEATURES_SHORT = [
```

```
    # Autoregressive (valid at short horizon - ACF > 0.6 at h=30d)
    'lag_price_1d', 'lag_price_7d', 'lag_price_30d', 'lag_price_365d',
    'ma_price_7d',  'ma_price_30d',
    'std_price_7d', 'std_price_30d',
    # Rolling statistics
    'roll_price_mean_7d',  'roll_price_std_7d',
    'roll_price_mean_14d', 'roll_price_std_14d',
    'roll_price_mean_30d', 'roll_price_std_30d',
    'roll_price_mean_90d', 'roll_price_std_90d',
    'roll_hydro_mean_7d',  'roll_hydro_mean_30d',
    'roll_gas_mean_7d',    'roll_gas_mean_30d',
    'price_momentum_7d',   'price_momentum_30d',
    # Hydro fundamentals (actual, available at forecast time)
    'hydro_reserve_gwh', 'inflow_hbv_gwh', 'inflow_snow_gwh',
    'reservoir_deviation_gwh', 'derived_inflow_z_score',
    'derived_hydro_change', 'hydro_fill_rate', 'snow_fraction',
    # Interactions
    'interaction_hydro_gas', 'ratio_gas_to_hydro',
    # Fuel prices (current front-month)
    'fuel_gas_eur_mwh', 'fuel_carbon_eur_t', 'fuel_brent_usd_bbl',
    'derived_gas_volatility_30d', 'gas_co2_ratio',
    # Forward / market expectations
    'forward_M1', 'forward_Y1',
    'epad_M1',    'epad_Y1',
    'derived_forward_slope', 'epad_slope',
    'implied_spot_M1', 'implied_spot_Y1', 'spot_fwd_basis',
    # Weather (1-day-ahead forecast available)
    'weather_temp_c', 'weather_wind_ms', 'weather_precip_mm',
    # Load and macro
    'load_mw', 'macro_eur_nok',
    # Calendar
    'cal_month_sin', 'cal_month_cos',
    'cal_day_of_year_sin', 'cal_day_of_year_cos',
    'season_winter', 'season_spring', 'season_summer', 'season_autumn',
    'is_weekend', 'month_num', 'week_of_year',
]

FEATURES_MEDIUM = [
    # Seasonal expectations (REPLACE raw AR lags at medium horizon)
    'seasonal_spot_price_mean',  'seasonal_spot_price_std',
    'seasonal_spot_price_p10',   'seasonal_spot_price_p90',
    'price_vs_seasonal',         'price_seasonal_z',
    # Hydro seasonal signals
    'seasonal_hydro_reserve_gwh_mean', 'seasonal_hydro_reserve_gwh_std',
    'seasonal_hydro_reserve_gwh_p10',  'seasonal_hydro_reserve_gwh_p90',
    'seasonal_inflow_hbv_gwh_mean',    'seasonal_inflow_hbv_gwh_std',
    # Current hydro levels (still known at forecast time)
    'hydro_reserve_gwh', 'reservoir_deviation_gwh',
    'derived_inflow_z_score', 'inflow_hbv_gwh',
    'hydro_seasonal_z', 'inflow_seasonal_z',
    # Forward market (still informative at 1-6 month horizon)
    'forward_M1', 'forward_Y1',
    'epad_M1',    'epad_Y1',
    'derived_forward_slope', 'epad_slope',
    'implied_spot_M1', 'implied_spot_Y1',
```

```
    # Fuel (front-month futures - informative up to ~6 months)
    'fuel_gas_eur_mwh', 'fuel_carbon_eur_t',
    'derived_gas_volatility_30d',
    'roll_gas_mean_30d',
    # Interaction
    'interaction_hydro_gas',
    # Macro
    'macro_eur_nok',
    # Calendar (now more important as horizon grows)
    'cal_month_sin', 'cal_month_cos',
    'cal_day_of_year_sin', 'cal_day_of_year_cos',
    'season_winter', 'season_spring', 'season_summer', 'season_autumn',
    'month_num', 'week_of_year',
    # Weak long-memory signal (flagged - may still help)
    'lag_price_365d',
    # NOTE: lag_price_1d, lag_price_7d, ma_price_7d are EXCLUDED
    #       (ACF < 0.3 at h=90d, unit-root spurious regression risk)
]

FEATURES_LONG = [
    # Forward Y+1 only (matched to long horizon, per Paper 3 finding)
    'forward_Y1', 'epad_Y1', 'implied_spot_Y1',
    # Seasonal hydro expectations (what we expect at this time of year)
    'seasonal_hydro_reserve_gwh_mean', 'seasonal_hydro_reserve_gwh_std',
    'seasonal_inflow_hbv_gwh_mean',
    # Seasonal price expectations
    'seasonal_spot_price_mean', 'seasonal_spot_price_std',
    'seasonal_spot_price_p10',  'seasonal_spot_price_p90',
    # Macro (long-run structural)
    'macro_eur_nok',
    # Calendar (primary stabilising signal at this horizon)
    'cal_month_sin', 'cal_month_cos',
    'cal_day_of_year_sin', 'cal_day_of_year_cos',
    'season_winter', 'season_spring', 'season_summer', 'season_autumn',
    'month_num', 'week_of_year',
    # NOTE: ALL autoregressive features EXCLUDED
    #       forward_M1 EXCLUDED (too short horizon, becomes spurious)
    #       fuel_gas, fuel_carbon EXCLUDED (unit root, spurious at 6-12mo)
]

# Filter to only columns that actually exist in df
FEATURES_SHORT  = [f for f in FEATURES_SHORT  if f in df.columns]
FEATURES_MEDIUM = [f for f in FEATURES_MEDIUM if f in df.columns]
FEATURES_LONG   = [f for f in FEATURES_LONG   if f in df.columns]

print("Horizon-Specific Feature Sets:")
print("="*60)
print(f"  SHORT-TERM  (h <= 30 days)  :  {len(FEATURES_SHORT):>3} features")
print(f"  MEDIUM-TERM (h = 1-6 months):  {len(FEATURES_MEDIUM):>3} features")
print(f"  LONG-TERM   (h = 6-12 months): {len(FEATURES_LONG):>3} features")
print("="*60)
print()
print("Key features EXCLUDED at each tier:")
print()
print("MEDIUM excludes:")
```

```
excluded_med = set(FEATURES_SHORT) - set(FEATURES_MEDIUM)
for f in sorted(excluded_med)[:15]:
    print(f"  - {f}")
print()
print("LONG additionally excludes:")
excluded_long = set(FEATURES_MEDIUM) - set(FEATURES_LONG)
for f in sorted(excluded_long):
    print(f"  - {f}")

ALL_HORIZONS = [1, 7, 14, 30, 60, 90, 180, 360]

# Add forward-shifted target columns (future spot price at each horizon)
for h in ALL_HORIZONS:
    df[f'target_h{h}'] = df['spot_price'].shift(-h)

target_cols = [f'target_h{h}' for h in ALL_HORIZONS]
meta_cols   = ['date', 'spot_price', 'regime', 'split',
               'year', 'month', 'iso_week', 'dow']

# Build final feature matrices for each horizon tier
def build_matrix(df, feature_cols, label):
    keep = meta_cols + feature_cols + target_cols
    keep = [c for c in keep if c in df.columns]
    mat = df[keep].copy()
    # Drop rows where ALL targets are NaN (end of series)
    mat = mat[mat[target_cols].notna().any(axis=1)]
    print(f"  {label:<18}: {mat.shape[0]} rows x {mat.shape[1]} columns")
    return mat

print("Building feature matrices...")
print()
df_short  = build_matrix(df, FEATURES_SHORT,  'Short-term')
df_medium = build_matrix(df, FEATURES_MEDIUM, 'Medium-term')
df_long   = build_matrix(df, FEATURES_LONG,   'Long-term')

print()
print("Checking for missing values in each matrix:")
for name, mat, feats in [
    ('Short',  df_short,  FEATURES_SHORT),
    ('Medium', df_medium, FEATURES_MEDIUM),
    ('Long',   df_long,   FEATURES_LONG),
]:
    feat_missing = mat[feats].isnull().sum()
    n_missing    = (feat_missing > 0).sum()
    print(f"  {name}: {n_missing} features with missing values")
    if n_missing > 0:
        print(feat_missing[feat_missing > 0].to_string())

print()

# Feature Importance Analysis
# We use Spearman rank correlation (not Pearson) because electricity prices have
# heavy tails from the crisis - Spearman is more robust.

def compute_importance(df, feature_cols, target='spot_price'):
```

```
    """
    Compute Spearman correlation of each feature with the target.
    Spearman is preferred over Pearson because:
      - It is rank-based and more robust to outliers (the extreme 2022 prices)
      - It captures monotone non-linear relationships
    """
    results = []
    y = df[target].dropna()
    for col in feature_cols:
        if col not in df.columns:
            continue
        x     = df.loc[y.index, col]
        valid = ~(x.isna() | y.isna())
        if valid.sum() < 10:
            continue
        rho, p = spearmanr(x[valid], y[valid])
        results.append({
            'feature':   col,
            'rho':       round(rho,  4),
            'abs_rho':   round(abs(rho), 4),
            'p_value':   round(p, 4),
            'direction': 'positive' if rho > 0 else 'negative',
        })
    out = (pd.DataFrame(results)
             .sort_values('abs_rho', ascending=False)
             .reset_index(drop=True))
    out.index += 1   # rank starts at 1
    return out

print("Computing feature importance (Spearman rho with spot_price)...")
print()

imp_short  = compute_importance(df_short,  FEATURES_SHORT)
imp_medium = compute_importance(df_medium, FEATURES_MEDIUM)
imp_long   = compute_importance(df_long,   FEATURES_LONG)

print("TOP 15 - SHORT-TERM features:")
print(imp_short[['feature','rho','abs_rho','direction']].head(15).to_string())
print()
print("TOP 15 - MEDIUM-TERM features:")
print(imp_medium[['feature','rho','abs_rho','direction']].head(15).to_string())
print()
print("TOP 15 - LONG-TERM features:")
print(imp_long[['feature','rho','abs_rho','direction']].head(15).to_string())

fig, axes = plt.subplots(3, 1, figsize=(10, 16))
fig.suptitle('Feature Importance by Horizon Tier\n'
             '(Spearman rho with spot_price)',
             fontsize=13, fontweight='bold')

tier_info = [
    (axes[0], imp_short,  'SHORT-TERM (h <= 30d)',    COLORS['short'],  20),
    (axes[1], imp_medium, 'MEDIUM-TERM (h = 1-6mo)',  COLORS['medium'], 20),
    (axes[2], imp_long,   'LONG-TERM (h = 6-12mo)',   COLORS['long'],   15),
]
```

```
535
536 for ax, imp_df, title, base_color, n_top in tier_info:
537     top = imp_df.head(n_top).sort_values('rho')   # ascending so highest is at top
538     bar_colors = [COLORS['crisis'] if v < 0 else base_color for v in top['rho']]
539     bars = ax.barh(range(len(top)), top['rho'], color=bar_colors, alpha=0.82)
540     ax.set_yticks(range(len(top)))
541     ax.set_yticklabels(top['feature'], fontsize=7.5)
542     ax.axvline(0, color='k', lw=1)
543     ax.set_xlabel('Spearman rho')
544     ax.set_title(title, fontsize=11, fontweight='bold')
545     ax.set_xlim(-1, 1)
546     # Add value labels
547     for bar, val in zip(bars, top['rho']):
548         offset = 0.02 if val >= 0 else -0.02
549         ha = 'left' if val >= 0 else 'right'
550         ax.text(val + offset, bar.get_y() + bar.get_height() / 2,
551                 f'{val:.3f}', ha=ha, va='center', fontsize=7)
552
553 plt.tight_layout()
554 plt.show()
555
556 # Hydro-Gas Interaction Visualisation
557 import matplotlib.pyplot as plt
558 import matplotlib.gridspec as gridspec
559 from scipy.stats import spearmanr
560
561 fig = plt.figure(figsize=(13, 10), constrained_layout=False)
562
563 fig.suptitle(
564     'Hydro-Gas Interaction: Norwegian Non-Linearity\n',
565     fontsize=14,
566     fontweight='bold',
567     y=0.97
568 )
569
570 # Top row only: 2 columns, normal spacing
571 gs = gridspec.GridSpec(
572     1, 2,
573     figure=fig,
574     left=0.07, right=0.95, top=0.89, bottom=0.57,
575     wspace=0.28
576 )
577
578 ax  = fig.add_subplot(gs[0, 0])   # Panel A
579 ax2 = fig.add_subplot(gs[0, 1])   # Panel B
580
581 # Panel C manually centered below with SAME width/height as A/B
582 posA = ax.get_position()
583 panel_width  = posA.width
584 panel_height = posA.height
585
586 x_center = 0.5 - panel_width / 2
587 y_bottom = 0.18   # adjust if needed
588
589 ax3 = fig.add_axes([x_center, y_bottom, panel_width, panel_height])
```

```
590
591 # -------------------------
592 # Panel A: Gas x Hydro => Spot Price
593 # -------------------------
594 sc = ax.scatter(
595     df['fuel_gas_eur_mwh'],
596     df['hydro_reserve_gwh'] / 1000,
597     c=df['spot_price'],
598     cmap='RdYlGn_r',
599     s=10,
600     alpha=0.6,
601     vmin=0,
602     vmax=300
603 )
604
605 ax.axhline(
606     df['hydro_reserve_gwh'].quantile(0.2) / 1000,
607     color='red', ls='--', lw=1.2, label='P20 hydro'
608 )
609 ax.axvline(
610     df['fuel_gas_eur_mwh'].quantile(0.8),
611     color='red', ls=':', lw=1.2, label='P80 gas'
612 )
613
614 ax.set_xlabel('Gas Price (EUR/MWh)')
615 ax.set_ylabel('Hydro Reserve (TWh)')
616 ax.set_title('Panel A - Gas × Hydro and Spot Price', fontsize=12,
617 fontweight='bold')
618 ax.legend(fontsize=8, loc='upper right')
619
620 cbar1 = fig.colorbar(sc, ax=ax, fraction=0.04, pad=0.02)
621 cbar1.set_label('Spot Price (EUR/MWh)')
622
623 # -------------------------
624 # Panel B: Interaction feature vs Spot
625 # -------------------------
626 valid = df[['interaction_hydro_gas', 'spot_price']].dropna()
627 rho_int, _ = spearmanr(valid['interaction_hydro_gas'], valid['spot_price'])
628
629 sc2 = ax2.scatter(
630     df['interaction_hydro_gas'],
631     df['spot_price'],
632     c=df['derived_gas_volatility_30d'],
633     cmap='Reds',
634     s=10,
635     alpha=0.55
636 )
637
638 ax2.axvline(0, color='gray', ls='--', lw=1)
639
640 ax2.set_xlabel('Interaction Feature (Z hydro × Z gas)')
641 ax2.set_ylabel('Spot Price (EUR/MWh)')
642 ax2.set_title(
643     f'Panel B - Interaction Feature vs Spot\nSpearman rho = {rho_int:.3f}',
644     fontsize=12,
```

```
    fontweight='bold'
)

ax2.text(
    0.03, 0.97,
    'Negative values = high hydro\n(low price expected)',
    transform=ax2.transAxes,
    fontsize=8,
    va='top',
    bbox=dict(fc='white', ec='gray', alpha=0.85)
)
ax2.text(
    0.62, 0.97,
    'Positive = low hydro\n(high price expected)',
    transform=ax2.transAxes,
    fontsize=8,
    va='top',
    bbox=dict(fc='white', ec='gray', alpha=0.85)
)

cbar2 = fig.colorbar(sc2, ax=ax2, fraction=0.04, pad=0.02)
cbar2.set_label('Gas Volatility (30d)')

# -------------------------
# Panel C: Hydro deviation from seasonal normal by regime
# -------------------------
regime_colors = {
    'Pre-Crisis (2018-mid2021)': COLORS['pre'],
    'Crisis (mid2021-2022)': COLORS['crisis'],
    'Post-Crisis (2023-2025)': COLORS['post'],
}

for regime, grp in df.groupby('regime', sort=False):
    if 'hydro_seasonal_z' not in grp.columns:
        continue

    ax3.scatter(
        grp['hydro_seasonal_z'],
        grp['spot_price'],
        color=regime_colors.get(regime, 'gray'),
        alpha=0.35,
        s=10,
        label=regime.split('(')[0].strip()
    )

ax3.axvline(0, color='gray', ls='--', lw=1, label='Seasonal normal')
ax3.set_xlabel('Hydro Z-score (sigma from seasonal mean)')
ax3.set_ylabel('Spot Price (EUR/MWh)')
ax3.set_title('Panel C - Hydro Deviation from Seasonal Normal', fontsize=12,
fontweight='bold')
ax3.legend(fontsize=8, loc='upper right')

for a in [ax, ax2, ax3]:
    a.grid(True, alpha=0.25)
plt.show()
```

```
700
701 fig, ax = plt.subplots(figsize=(14, 7))
702 fig.suptitle('Engineered Feature Relevance Decay\n',
703              fontsize=13, fontweight='bold')
704
705 horizons = [1, 7, 14, 30, 60, 90, 120, 180, 270, 360]
706
707 # Select the most representative features from each tier
708 engineered_features = {
709     # Short-tier leaders
710     'lag_price_1d':           (COLORS['crisis'],  '--'),
711     'implied_spot_M1':        (COLORS['pre'],     '-'),
712     'interaction_hydro_gas':  (COLORS['hydro'],   '-'),
713     'price_vs_seasonal':      (COLORS['gas'],     '-'),
714     # Medium-tier leaders
715     'forward_M1':             ('#9C27B0',          '-'),
716     'forward_Y1':             ('#4CAF50',          '-'),
717     'hydro_seasonal_z':       ('#00BCD4',          '-'),
718     # Long-tier leaders
719     'implied_spot_Y1':        ('#795548',          '-'),
720     'seasonal_spot_price_mean':('#E91E63',         ':'),
721 }
722
723 for feat, (color, ls) in engineered_features.items():
724     if feat not in df.columns:
725         continue
726     corrs = []
727     for h in horizons:
728         if h >= len(df):
729             corrs.append(np.nan)
730             continue
731         shifted = df['spot_price'].shift(-h)
732         valid   = ~(shifted.isna() | df[feat].isna())
733         if valid.sum() < 20:
734             corrs.append(np.nan)
735             continue
736         rho, _ = spearmanr(df.loc[valid, feat], shifted[valid])
737         corrs.append(abs(rho))
738     ax.plot(horizons, corrs, 'o-', lw=2, ms=5, ls=ls,
739             color=color, label=feat)
740
741 # Shade the three horizon zones
742 ax.axvspan(0,   30,  alpha=0.06, color='blue',   label='Short zone')
743 ax.axvspan(30,  180, alpha=0.06, color='orange', label='Medium zone')
744 ax.axvspan(180, 370, alpha=0.06, color='green',  label='Long zone')
745 ax.axvline(30,  color='gray', ls=':', lw=1.2)
746 ax.axvline(180, color='gray', ls=':', lw=1.2)
747 ax.text(15,  0.02, 'SHORT',  fontsize=9, ha='center', color='navy')
748 ax.text(100, 0.02, 'MEDIUM', fontsize=9, ha='center', color='darkorange')
749 ax.text(270, 0.02, 'LONG',   fontsize=9, ha='center', color='darkgreen')
750 ax.axhline(0.3, color='green',  ls='--', lw=1.0, alpha=0.7, label='0.3 threshold')
751 ax.axhline(0.1, color='orange', ls='--', lw=1.0, alpha=0.7, label='0.1 threshold')
752
753 ax.set_xlabel('Forecasting Horizon (days)')
754 ax.set_ylabel('|Spearman rho| with future spot price')
```

```
ax.legend(fontsize=8, ncol=3, loc='upper right')
ax.set_ylim(0, 1)
ax.set_xticks(horizons)
ax.set_xticklabels([f'{h}d' for h in horizons])

plt.tight_layout()
plt.show()

# Coefficient Bounds from Merit-Order Theory
import matplotlib.patches as mpatches

# ----------------------------------------
# FUNDAMENTAL COEFFICIENT BOUNDS - Norwegian Hydro Merit-Order Model
# Based on Ghelasi & Ziel (2025) Table 3, adapted for Norwegian hydro market
# ----------------------------------------
# In thermal markets: coefficient of gas price = 1/eta_gas (heat rate)
# In Norway: hydro coefficients are NEGATIVE (more water => lower price)
#            Gas coefficients retain same thermal upper bound (via
interconnectors)
# ----------------------------------------

COEF_BOUNDS = {
    # Continental fuel linkages (via interconnectors to DK/DE/NL/UK)
    'fuel_gas_eur_mwh':        (0,      4.00),   # max heat rate = 1/eta_gas
    = 1/0.25
    'fuel_carbon_eur_t':       (0,      1.33),   # max eps/eta  = 0.4/0.3
    (lignite)
    'fuel_brent_usd_bbl':      (0,      0.588),  # max oil heat rate

    # Norwegian hydro fundamentals (NEGATIVE: more water => lower price)
    'hydro_reserve_gwh':       (None,   0),
    'inflow_hbv_gwh':          (None,   0),
    'reservoir_deviation_gwh': (None,   0),
    'derived_inflow_z_score':  (None,   0),
    'hydro_seasonal_z':        (None,   0),
    'inflow_seasonal_z':       (None,   0),

    # Demand (POSITIVE: higher demand => higher price)
    'load_mw':                 (0,      None),

    # Forward prices (POSITIVE contribution to expected price)
    'forward_M1':              (0,      None),
    'forward_Y1':              (0,      None),
    'epad_M1':                 (0,      None),   # area premium >= 0
    'epad_Y1':                 (0,      None),
    'implied_spot_M1':         (0,      None),
    'implied_spot_Y1':         (0,      None),

    # Autoregressive (POSITIVE: price persistence)
    'lag_price_1d':            (0,      None),
    'lag_price_7d':            (0,      None),
    'lag_price_30d':           (0,      None),
    'lag_price_365d':          (0,      None),
    'price_vs_seasonal':       (0,      None),   # deviation above seasonal =>
    # higher price
```

```
}

# Display the constraint table
print("Coefficient Bounds - Norwegian Hydro Merit-Order Model")
print("(Analogous to Table 3 in Ghelasi & Ziel 2025)")
print()
print(f"{'Feature':<32}  {'Lower':<10}  {'Upper':<10}  {'Reasoning'}")
print("-"*75)
bounds_explained = {
    'fuel_gas_eur_mwh':         '1/eta_gas (max heat rate = 4.0)',
    'fuel_carbon_eur_t':        'eps/eta_lignite = 0.4/0.3 = 1.33',
    'fuel_brent_usd_bbl':       '1/eta_oil * conv = 0.588',
    'hydro_reserve_gwh':        'More water => lower price (merit order)',
    'inflow_hbv_gwh':           'More inflow => lower future water value',
    'reservoir_deviation_gwh': 'Above seasonal normal => lower scarcity',
    'derived_inflow_z_score':  'Higher inflow z => more surplus',
    'load_mw':                  'Higher demand => higher equilibrium price',
    'forward_M1':               'Market expectation (positive contribution)',
    'forward_Y1':               'Market expectation (positive contribution)',
    'epad_M1':                  'Area premium >= 0 by definition',
    'lag_price_1d':             'Price persistence (positive autocorrelation)',
    'price_vs_seasonal':        'Above seasonal expectation => price boost',
}
for feat, (lo, hi) in COEF_BOUNDS.items():
    lo_str = str(lo)   if lo is not None else '-inf'
    hi_str = str(hi)   if hi is not None else '+inf'
    reason = bounds_explained.get(feat, '')
    print(f"  {feat:<30}  [{lo_str:<8}, {hi_str:<8}]  {reason}")

print()
print("These bounds will be applied to the Constrained Elastic Net model.")

"""
Empirical Test: Forward Price Maturity Selection on NO1 Market
============================================================
Replicates the liquidity analysis of Ghelasi and Ziel (2025) on the
NO1 dataset using LassoLarsIC with BIC criterion.

For each forecasting horizon h, we fit a LassoLarsIC model with BIC
on a rolling expanding window and record how often M1 and Y1 are
selected, and their mean absolute coefficients when selected.

This directly tests whether M1 or Y1 dominates at each horizon,
following the methodology of Ghelasi and Ziel (2025).
"""

# - Load data ------------------------------
DATA_PATH = 'master_clean_NO1.csv'
df = pd.read_csv(DATA_PATH, parse_dates=['date'])
df = df.sort_values('date').reset_index(drop=True)

TRAIN_END = pd.Timestamp('2021-12-31')
VAL_START = pd.Timestamp('2022-01-01')
TEST_END  = pd.Timestamp('2024-12-31')

```

```
df['split'] = df['date'].apply(
    lambda d: 'train' if d <= TRAIN_END
    else ('eval' if d <= TEST_END else 'holdout'))

# - Control features (shared across all horizons) -------------
CONTROLS = [c for c in [
    'fuel_gas_eur_mwh', 'fuel_carbon_eur_t',
    'hydro_reserve_gwh', 'reservoir_deviation_gwh',
    'weather_temp_c', 'load_mw', 'macro_eur_nok',
    'cal_month_sin', 'cal_month_cos',
    'cal_day_of_year_sin', 'cal_day_of_year_cos',
    'season_winter', 'season_summer',
    'lag_price_365d',
] if c in df.columns]

# Forward candidates to compare
FWD_CANDIDATES = {
    'forward_M1':  'M1 (front-month)',
    'forward_Y1':  'Y1 (year-ahead)',
    'epad_M1':     'EPAD M1',
    'epad_Y1':     'EPAD Y1',
}

# - Rolling LassoLarsIC selection ---------------------
HORIZONS   = [30, 90, 180, 360]
ROLL_STEP  = 30
WINDOW     = 365 * 3   # 3-year rolling window

def run_selection(df, horizons, controls, fwd_candidates,
                  window=WINDOW, roll_step=ROLL_STEP):
    """
    For each horizon h and each rolling window step:
      1. Build X = controls + all forward candidates
      2. Build y = spot price h days ahead
      3. Fit LassoLarsIC (BIC)
      4. Record which forward candidates are selected (non-zero coef)
         and their absolute standardised coefficient
    Returns: DataFrame with selection results per horizon and candidate.
    """
    all_features = controls + list(fwd_candidates.keys())
    all_features = [c for c in all_features if c in df.columns]

    eval_mask  = (df['date'] >= VAL_START) & (df['date'] <= TEST_END)
    eval_dates = df.loc[eval_mask, 'date'].values[::roll_step]

    records = []

    for h in horizons:
        print(f"  Testing h={h}d...", end=' ')
        n_selected_m1 = 0
        n_selected_y1 = 0
        total_steps   = 0
        coef_sums     = {k: 0.0 for k in fwd_candidates}
        coef_counts   = {k: 0   for k in fwd_candidates}

```

```
        for fc_ts in eval_dates:
            fc_date  = pd.Timestamp(fc_ts)
            tgt_date = fc_date + pd.Timedelta(days=h)

            # Training window
            t_win = fc_date - pd.Timedelta(days=window)
            tr    = df[(df['date'] > t_win) & (df['date'] <= fc_date)].copy()

            # Target: price h days ahead from training window perspective
            # Use shifted target within training data
            tr = tr.copy()
            tr['target'] = tr['spot_price'].shift(-h)
            tr = tr.dropna(subset=['target'] + all_features)
            if len(tr) < 60:
                continue

            X = tr[all_features].fillna(0).values.astype(float)
            y = tr['target'].values.astype(float)

            scaler = StandardScaler()
            Xs = scaler.fit_transform(X)

            try:
                model = LassoLarsIC(criterion='bic', max_iter=200)
                model.fit(Xs, y)
                coefs = model.coef_
            except Exception:
                continue

            total_steps += 1

            for j, feat in enumerate(all_features):
                if feat in fwd_candidates:
                    abs_c = abs(coefs[j])
                    if abs_c > 1e-8:
                        coef_sums[feat]   += abs_c
                        coef_counts[feat] += 1

        print(f"{total_steps} steps")

        for feat, label in fwd_candidates.items():
            if feat not in all_features:
                records.append({'horizon': h, 'feature': feat,
                                'label': label,
                                'selection_rate': np.nan,
                                'mean_abs_coef': np.nan})
                continue
            sel_rate = coef_counts[feat] / max(total_steps, 1)
            records.append({
                'horizon':       h,
                'feature':       feat,
                'label':         label,
                'selection_rate': round(sel_rate, 4),
                'n_selected':    coef_counts[feat],
                'n_total':       total_steps,
```

```
            })

    return pd.DataFrame(records)


print("Running LassoLarsIC (BIC) forward maturity selection test...")
print(f"Controls: {len(CONTROLS)} variables")
print(f"Forward candidates: {list(FWD_CANDIDATES.keys())}")
print(f"Horizons: {HORIZONS}")
print()

results = run_selection(df, HORIZONS, CONTROLS, FWD_CANDIDATES)

# - Summary table ----------------------------
print()
print("="*70)
print("SELECTION RATE - fraction of rolling windows where feature is selected")
print("(LassoLarsIC BIC  |  higher = more consistently chosen)")
print("="*70)
pivot_rate = results.pivot_table(
    index='label', columns='horizon', values='selection_rate').round(3)
print(pivot_rate.to_string())

print()
print("="*70)
print("MEAN ABSOLUTE STANDARDISED COEFFICIENT (when selected)")
print("(higher = larger influence on the forecast)")
print("="*70)
pivot_coef = results.pivot_table(
    index='label', columns='horizon', values='mean_abs_coef').round(4)
print(pivot_coef.to_string())

results.to_csv('forward_maturity_selection_NO1.csv', index=False)
print("\nSaved: forward_maturity_selection_NO1.csv")

# - Figure: Selection rate by horizon -------------------
fig, axes = plt.subplots(1, 1, figsize=(14, 6))
fig.suptitle('Forward Price Maturity Selection - NO1 Market\n'
             'LassoLarsIC with BIC  |  Replication of Ghelasi and Ziel (2025)',
             fontsize=13, fontweight='bold')

colors = {'forward_M1': '#2196F3', 'forward_Y1': '#4CAF50',
          'epad_M1':    '#FF9800', 'epad_Y1':    '#9C27B0'}

x     = np.arange(len(HORIZONS))
width = 0.2
axes.set_title('Selection Rate')
plt.tight_layout()
plt.show()
```

Appendix E

Source Code for Baseline Models and Results

```
import warnings; warnings.filterwarnings('ignore')
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.patches import Patch
from scipy import stats
from scipy.stats import pearsonr, spearmanr

# Scikit-learn
from sklearn.linear_model import ElasticNet
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Statsmodels
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.stattools import acf

plt.rcParams.update({
    'figure.facecolor': 'white',  'axes.facecolor': '#f8f9fa',
    'axes.grid': True,            'grid.alpha': 0.35,
    'font.size': 11,              'axes.spines.top': False,
    'axes.spines.right': False,   'axes.labelsize': 12,
    'axes.titlesize': 13,         'figure.dpi': 110,
    'savefig.dpi': 150,           'savefig.bbox': 'tight',
})

COLORS = {
    'pre':    '#2196F3', 'crisis': '#F44336', 'post':   '#4CAF50',
    'hydro':  '#00BCD4', 'gas':    '#FF9800', 'fwd':    '#9C27B0',
    'short':  '#1976D2', 'medium': '#F57C00', 'long':   '#388E3C',
}

# Regime / split boundaries
CRISIS_START = pd.Timestamp('2021-07-01')
CRISIS_END   = pd.Timestamp('2023-01-01')
TRAIN_END    = pd.Timestamp('2021-12-31')
VAL_START    = pd.Timestamp('2022-01-01')
```

```
VAL_END      = pd.Timestamp('2022-12-31')
TEST_START   = pd.Timestamp('2023-01-01')
TEST_END     = pd.Timestamp('2024-12-31')

# Model colour palette (consistent across all plots)
MODEL_COLORS = {
    'Naive':                    '#9E9E9E',
    'Seasonal-WD':              '#607D8B',
    'SARIMA':                   '#2196F3',
    'ElasticNet-Unconstrained': '#FF9800',
    'ElasticNet-Constrained':   '#4CAF50',
}
MODEL_MARKERS = {
    'Naive': 'o', 'Seasonal-WD': 's', 'SARIMA': '^',
    'ElasticNet-Unconstrained': 'D', 'ElasticNet-Constrained': '*',
}
MONTHS = ['Jan','Feb','Mar','Apr','May','Jun',
          'Jul','Aug','Sep','Oct','Nov','Dec']

DATA_PATH = 'master_clean_NO1.csv'   # <-- update to your path

df = pd.read_csv(DATA_PATH, parse_dates=['date'])
df = df.sort_values('date').reset_index(drop=True)

# - Regime and split labels ------------------------
def get_regime(d):
    if d < CRISIS_START:  return 'Pre-Crisis'
    elif d < CRISIS_END:  return 'Crisis'
    return 'Post-Crisis'

def get_split(d):
    if d <= TRAIN_END:              return 'train'
    elif VAL_START <= d <= VAL_END: return 'val'
    elif TEST_START <= d <= TEST_END: return 'test'
    return 'holdout'

df['regime']   = df['date'].apply(get_regime)
df['split']    = df['date'].apply(get_split)
df['year']     = df['date'].dt.year
df['month']    = df['date'].dt.month
df['dow']      = df['date'].dt.dayofweek
df['iso_week'] = df['date'].dt.isocalendar().week.astype(int)

# - Seasonal profiles (from training data only) -------------
train = df[df['date'] <= TRAIN_END].copy()

def seasonal_profile(df_train, col):
    prof = (df_train.groupby(['month','iso_week'])[col]
            .agg(s_mean='mean', s_std='std',
                 s_p10=lambda x: np.percentile(x,10),
                 s_p90=lambda x: np.percentile(x,90))
            .reset_index())
    prof['s_mean'] = prof['s_mean'].rolling(3, center=True, min_periods=1).mean()
    return prof

```

```
prof_spot  = seasonal_profile(train, 'spot_price')
prof_hydro = seasonal_profile(train, 'hydro_reserve_gwh')
prof_inflow= seasonal_profile(train, 'inflow_hbv_gwh')

for col, prof, prefix in [
    ('spot_price',        prof_spot,   'sp'),
    ('hydro_reserve_gwh', prof_hydro,  'hy'),
    ('inflow_hbv_gwh',    prof_inflow, 'if'),
]:
    merged = df[['month','iso_week']].merge(
        prof[['month','iso_week','s_mean','s_std','s_p10','s_p90']],
        on=['month','iso_week'], how='left')
    df[f'seasonal_{col}_mean'] = merged['s_mean'].values
    df[f'seasonal_{col}_std']  = merged['s_std'].values
    df[f'seasonal_{col}_p10']  = merged['s_p10'].values
    df[f'seasonal_{col}_p90']  = merged['s_p90'].values

# - Derived features ---------------------------
df['price_vs_seasonal']  = df['spot_price'] - df['seasonal_spot_price_mean']
df['price_seasonal_z']   = (df['price_vs_seasonal']
                            / df['seasonal_spot_price_std'].clip(lower=1))
df['hydro_seasonal_z']   = ((df['hydro_reserve_gwh']
                             - df['seasonal_hydro_reserve_gwh_mean'])
                             / df['seasonal_hydro_reserve_gwh_std'].clip(lower=1))
df['implied_spot_M1']    = df['forward_M1'] + df['epad_M1']
df['implied_spot_Y1']    = df['forward_Y1'] + df['epad_Y1']
df['epad_slope']         = df['epad_M1'] - df['epad_Y1']

hydro_z = (df['hydro_reserve_gwh'] - df['hydro_reserve_gwh'].mean()) /
df['hydro_reserve_gwh'].std()
gas_z   = (df['fuel_gas_eur_mwh']  - df['fuel_gas_eur_mwh'].mean())  /
df['fuel_gas_eur_mwh'].std()
df['interaction_hydro_gas'] = hydro_z * gas_z

for w in [7, 14, 30, 90]:
    df[f'roll_price_mean_{w}d'] = df['spot_price'].shift(1).rolling(w).mean()
    df[f'roll_price_std_{w}d']  = df['spot_price'].shift(1).rolling(w).std()

m = df['date'].dt.month
df['season_winter'] = ((m<=2)|(m==12)).astype(int)
df['season_spring'] = ((m>=3)&(m<=5)).astype(int)
df['season_summer'] = ((m>=6)&(m<=8)).astype(int)
df['season_autumn'] = ((m>=9)&(m<=11)).astype(int)
df['is_weekend']    = (df['dow']>=5).astype(int)
df['month_num']     = df['date'].dt.month
df['week_of_year']  = df['iso_week']

print(f"Data loaded: {len(df)} rows | {df['date'].min().date()} to {df['date']
.max().date()}")
print(f"Train: {(df['split']=='train').sum()} | Val: {(df['split']=='val')
.sum()}|"
      f"Test: {(df['split']=='test').sum()} | Holdout: {(df['split']=='holdout')
      .sum()}")

# - Horizon-specific feature columns -------------------
```

```
FEATURES_SHORT = [c for c in [
    'lag_price_1d','lag_price_7d','lag_price_30d','lag_price_365d',
    'ma_price_7d','ma_price_30d','std_price_7d','std_price_30d',
    'roll_price_mean_7d','roll_price_std_7d','roll_price_mean_30d',
    'roll_price_std_30d',
    'roll_price_mean_90d','roll_price_std_90d',
    'hydro_reserve_gwh','inflow_hbv_gwh','reservoir_deviation_gwh',
    'derived_inflow_z_score','interaction_hydro_gas',
    'fuel_gas_eur_mwh','fuel_carbon_eur_t','derived_gas_volatility_30d',
    'forward_M1','forward_Y1','epad_M1','epad_Y1',
    'derived_forward_slope','epad_slope','implied_spot_M1','implied_spot_Y1',
    'weather_temp_c','weather_wind_ms','weather_precip_mm','load_mw',
    'macro_eur_nok','cal_month_sin','cal_month_cos',
    'cal_day_of_year_sin','cal_day_of_year_cos',
    'season_winter','season_spring','season_summer','season_autumn',
    'is_weekend','month_num','week_of_year',
] if c in df.columns]

FEATURES_MEDIUM = [c for c in [
    'seasonal_spot_price_mean','seasonal_spot_price_std',
    'seasonal_spot_price_p10','seasonal_spot_price_p90',
    'price_vs_seasonal','price_seasonal_z',
    'seasonal_hydro_reserve_gwh_mean','seasonal_hydro_reserve_gwh_std',
    'seasonal_inflow_hbv_gwh_mean',
    'hydro_reserve_gwh','reservoir_deviation_gwh',
    'derived_inflow_z_score','hydro_seasonal_z',
    'forward_M1','forward_Y1','epad_M1','epad_Y1',
    'epad_slope','implied_spot_M1','implied_spot_Y1',
    'fuel_gas_eur_mwh','fuel_carbon_eur_t','derived_gas_volatility_30d',
    'interaction_hydro_gas','macro_eur_nok','lag_price_365d',
    'cal_month_sin','cal_month_cos','cal_day_of_year_sin','cal_day_of_year_cos',
    'season_winter','season_spring','season_summer','season_autumn',
    'month_num','week_of_year',
] if c in df.columns]

FEATURES_LONG = [c for c in [
    'forward_Y1','epad_Y1','implied_spot_Y1',
    'seasonal_hydro_reserve_gwh_mean','seasonal_hydro_reserve_gwh_std',
    'seasonal_inflow_hbv_gwh_mean',
    'seasonal_spot_price_mean','seasonal_spot_price_std',
    'seasonal_spot_price_p10','seasonal_spot_price_p90',
    'macro_eur_nok','cal_month_sin','cal_month_cos',
    'cal_day_of_year_sin','cal_day_of_year_cos',
    'season_winter','season_spring','season_summer','season_autumn',
    'month_num','week_of_year',
] if c in df.columns]

# Coefficient bounds (Norwegian hydro merit-order)
COEF_BOUNDS = {
    'fuel_gas_eur_mwh':        (0, 4.0),
    'fuel_carbon_eur_t':       (0, 1.33),
    'fuel_brent_usd_bbl':      (0, 0.588),
    'hydro_reserve_gwh':       (None, 0),
    'inflow_hbv_gwh':          (None, 0),
    'reservoir_deviation_gwh': (None, 0),
```

```
    'derived_inflow_z_score':  (None, 0),
    'hydro_seasonal_z':        (None, 0),
    'load_mw':                 (0, None),
    'forward_M1':              (0, None),
    'forward_Y1':              (0, None),
    'epad_M1':                 (0, None),
    'epad_Y1':                 (0, None),
    'implied_spot_M1':         (0, None),
    'implied_spot_Y1':         (0, None),
    'lag_price_1d':            (0, None),
    'lag_price_7d':            (0, None),
    'lag_price_30d':           (0, None),
    'lag_price_365d':          (0, None),
    'price_vs_seasonal':       (0, None),
    'price_seasonal_z':        (0, None),
}

print(f"\nFeature set sizes:")

# Model 1: Naive Model
class NaiveModel:
    """
    Naive Benchmark - Weekday-Preserving Random Walk

    Forecast logic:
      h = 1  day  : predict last known price (pure random walk)
      h = 2-7 days: predict last known price (carry forward)
      h > 7  days : predict average price for the same weekday
                    observed over the past 90 training days
    """

    name = 'Naive'

    def fit(self, train_df):
        self.last_price    = float(train_df['spot_price'].iloc[-1])
        self.weekday_means = (train_df
                              .groupby(train_df['date'].dt.dayofweek)
                              ['spot_price'].mean().to_dict())
        self.overall_mean  = float(train_df['spot_price'].mean())
        # Residual std from 1-day naive errors on training data
        naive_1d = train_df['spot_price'].shift(1)
        resids   = train_df['spot_price'] - naive_1d
        self.resid_std = float(resids.dropna().std())
        return self

    def predict(self, horizon, forecast_date):
        if horizon <= 7:
            return self.last_price
        target_dow = (forecast_date + pd.Timedelta(days=horizon)).dayofweek
        return self.weekday_means.get(target_dow, self.overall_mean)


# - Quick demonstration --------------------------
train_demo = df[df['split'] == 'train'].copy()
m = NaiveModel().fit(train_demo)
```

```
260
261 print("Naive model fitted on training data.")
262 print(f"  Last known price : {m.last_price:.2f} EUR/MWh")
263 print(f"  Weekday means (Mon-Sun):")
264 for dow, name in enumerate(['Mon','Tue','Wed','Thu','Fri','Sat','Sun']):
265     print(f"    {name}: {m.weekday_means.get(dow, 0):.2f} EUR/MWh")
266 print(f"  Training residual std : {m.resid_std:.2f} EUR/MWh")
267 print()
268 print("Prediction examples:")
269 fc_date = pd.Timestamp('2023-06-01')
270 for h in [1, 7, 30, 90, 180, 360]:
271     pred = m.predict(h, fc_date)
272     print(f"  h={h:3d}d ahead from {fc_date.date()} => {pred:.2f} EUR/MWh")
273
274 # Model 2: Seasonal Weekday-Season (WD)
275 class SeasonalWDModel:
276     """
277     Seasonal Weekday-Season Benchmark (WD model)
278
279     Predicts the historical average price for the (weekday, season) pair
280     of the target date, computed from the training window.
281
282     Seasons: Winter=DJF, Spring=MAM, Summer=JJA, Autumn=SON
283
284     This is a pure seasonality model - it knows nothing about
285     recent price levels or fundamental drivers. Its purpose is to
286     establish a lower bound: any useful model must beat this
287     by capturing price variation beyond seasonal patterns.
288     """
289
290     name = 'Seasonal-WD'
291
292     @staticmethod
293     def _season(month):
294         return {12:'W',1:'W',2:'W', 3:'Sp',4:'Sp',5:'Sp',
295                 6:'Su',7:'Su',8:'Su', 9:'A',10:'A',11:'A'}[month]
296
297     def fit(self, train_df):
298         df_t = train_df.copy()
299         df_t['dow']    = df_t['date'].dt.dayofweek
300         df_t['season'] = df_t['date'].dt.month.map(self._season)
301         self.lookup    = (df_t.groupby(['dow','season'])['spot_price']
302                           .mean().to_dict())
303         self.fallback  = float(train_df['spot_price'].mean())
304         self.resid_std = float(train_df['spot_price'].std())
305         return self
306
307     def predict(self, horizon, forecast_date):
308         target_date = forecast_date + pd.Timedelta(days=horizon)
309         key = (target_date.dayofweek, self._season(target_date.month))
310         return self.lookup.get(key, self.fallback)
311
312
313 # - Quick demonstration -------------------------
314 m_wd = SeasonalWDModel().fit(train_demo)
```

```
315
316 print("Seasonal-WD model fitted.")
317 print()
318 print("Seasonal averages by (weekday, season) - first 12 entries:")
319 items = list(m_wd.lookup.items())[:12]
320 for (dow, season), price in items:
321     day_name = ['Mon','Tue','Wed','Thu','Fri','Sat','Sun'][dow]
322     print(f"  {day_name} / {season:<3} : {price:.2f} EUR/MWh")
323 print()
324
325 # Show that WD gives identical answer for any horizon with same weekday/season
326 print("WD predictions (same weekday+season = same prediction at any horizon):")
327 fc_date = pd.Timestamp('2023-06-01')
328 for h in [1, 7, 30, 90, 180, 360]:
329     pred = m_wd.predict(h, fc_date)
330     target = fc_date + pd.Timedelta(days=h)
331     print(f"  h={h:3d}d => target {target.date()} "
332           f"({['Mon','Tue','Wed','Thu','Fri','Sat','Sun'][target.dayofweek]}, "
333           f"{SeasonalWDModel._season(target.month)}) => {pred:.2f} EUR/MWh")
334
335 print()
336
337 # Model 3: SARIMA(2,1,1)(1,0,1)[7]
338 class SARIMAModel:
339     """
340     SARIMA(2,1,1)(1,0,1)[7] - Seasonal AutoRegressive Integrated Moving Average
341
342     Order selection rationale (from NB1 PACF analysis):
343       - PACF significant at lags 1, 2, 7  => AR(2) + seasonal AR(1) at s=7
344       - Series needs differencing (unit root confirmed in NB1) => d=1
345       - MA(1) + seasonal MA(1) for residual structure => MA(1), Q=1
346       - Weekly seasonality period s=7 (daily data)
347
348     This is the standard EPF benchmark for time-series-only models.
349     It captures:
350       - Short-term autocorrelation (AR and MA terms)
351       - Weekly seasonal pattern (SAR and SMA at s=7)
352       - Trend removal via first differencing
353
354     Limitation: SARIMA only uses the price series itself.
355     It ignores all fundamental drivers (gas, hydro, forwards).
356     SARIMAX would add exogenous variables, but SARIMA gives a clean
357     baseline of what autocorrelation structure alone can achieve.
358
359     Re-estimation: For the rolling evaluation, SARIMA is re-estimated
360     every ROLL_STEP days on the expanding training window.
361     """
362
363     name = 'SARIMA'
364
365     def __init__(self, order=(2,1,1), seasonal_order=(1,0,1,7)):
366         self.order          = order
367         self.seasonal_order = seasonal_order
368         self.fitted         = None
369         self.resid_std      = None
```

```
370
371     def fit(self, train_df):
372         price = (train_df.set_index('date')['spot_price']
373                  .asfreq('D').ffill())
374         model = SARIMAX(price,
375                         order=self.order,
376                         seasonal_order=self.seasonal_order,
377                         enforce_stationarity=False,
378                         enforce_invertibility=False)
379         self.fitted    = model.fit(disp=False, maxiter=80)
380         self.resid_std = float(self.fitted.resid.std())
381         return self
382
383     def predict(self, horizon):
384         if self.fitted is None:
385             raise RuntimeError("Call fit() first")
386         fc = self.fitted.get_forecast(steps=horizon)
387         return float(fc.predicted_mean.iloc[-1])
388
389     def predict_interval(self, horizon, alpha=0.05):
390         fc  = self.fitted.get_forecast(steps=horizon)
391         ci  = fc.conf_int(alpha=alpha)
392         return (float(fc.predicted_mean.iloc[-1]),
393                 float(ci.iloc[-1, 0]),
394                 float(ci.iloc[-1, 1]))
395
396
397 # - Fit on training data and show diagnostics --------------
398 print("Fitting SARIMA(2,1,1)(1,0,1)[7] on training data...")
399 print("(This takes ~20-30 seconds)")
400 t0 = time.time()
401 m_sarima = SARIMAModel().fit(train_demo)
402 print(f"Fitted in {time.time()-t0:.1f}s")
403 print()
404 print(f"AIC  : {m_sarima.fitted.aic:.2f}")
405 print(f"BIC  : {m_sarima.fitted.bic:.2f}")
406 print(f"Resid std : {m_sarima.resid_std:.2f} EUR/MWh")
407 print()
408 print("SARIMA predictions from end of training (h=1 to 360):")
409 for h in [1, 7, 30, 90, 180, 360]:
410     pt, lo, hi = m_sarima.predict_interval(h, alpha=0.05)
411     print(f"  h={h:3d}d : {pt:.2f} EUR/MWh  [95% PI: {lo:.1f}, {hi:.1f}]")
412 print()
413
414 # Models 4 & 5: Constrained and Unconstrained Elastic Net
415 from scipy.optimize import minimize
416
417 class ElasticNetModel:
418     """
419     Elastic Net Regression - Linear model with L1 + L2 regularisation.
420
421     Objective:
422         min_{beta} ||y - X*beta||^2 + alpha*l1_ratio*||beta||_1
423                                    + alpha*(1-l1_ratio)/2*||beta||^2
424
```

```
    Two modes:
    -----
    mode='unconstrained':
        Standard sklearn ElasticNet. No physical constraints.
        Can produce negative gas coefficients or positive hydro coefficients
        if the training data is noisy or multicollinear.

    mode='constrained':
        Adds the Norwegian hydro merit-order coefficient bounds from NB2.
        Solved via L-BFGS-B optimisation (handles box constraints).
        Prevents economically nonsensical coefficients.

    Why does constraining help?
        At longer horizons, multicollinearity and unit-root spurious regression
        can push coefficients in the wrong direction. Physical bounds act as
        a regulariser that keeps the model economically interpretable.
        Ghelasi & Ziel (2025) show constrained models outperform unconstrained
        by 18-29% RMSE at medium/long horizons.

    Feature set selection (automatic by horizon):
        h <= 30  days : FEATURES_SHORT
        h = 31-180 days : FEATURES_MEDIUM
        h > 180  days : FEATURES_LONG
    """

    def __init__(self, alpha=0.5, l1_ratio=0.5, mode='unconstrained'):
        self.alpha    = alpha
        self.l1_ratio = l1_ratio
        self.mode     = mode
        self.name     = f'ElasticNet-{mode.capitalize()}'
        self.scaler   = StandardScaler()
        self.coef_    = None
        self.intercept_ = 0.0
        self.feature_cols_ = []
        self.resid_std_    = 1.0

    def _get_bounds(self, feature_cols, scales):
        bounds = []
        for col, s in zip(feature_cols, scales):
            lo, hi = COEF_BOUNDS.get(col, (None, None))
            lo_s = (-np.inf if lo is None else lo * s)
            hi_s = ( np.inf if hi is None else hi * s)
            bounds.append((lo_s, hi_s))
        return bounds

    def fit(self, X, y):
        self.feature_cols_ = list(X.columns)
        Xn = X.fillna(0).values.astype(float)
        yn = y.fillna(float(y.median())).values.astype(float)
        Xs = self.scaler.fit_transform(Xn)

        if self.mode == 'unconstrained':
            net = ElasticNet(alpha=self.alpha, l1_ratio=self.l1_ratio,
                             max_iter=2000, fit_intercept=True)
            net.fit(Xs, yn)
```

```
            self.coef_      = net.coef_
            self.intercept_ = net.intercept_

        else:  # constrained - L-BFGS-B with physical bounds
            bounds = self._get_bounds(self.feature_cols_,
                                      self.scaler.scale_)

            # Warm start: unconstrained solution clipped to bounds
            net0 = ElasticNet(alpha=self.alpha, l1_ratio=self.l1_ratio,
                              max_iter=2000, fit_intercept=True)
            net0.fit(Xs, yn)
            coef0 = np.clip(net0.coef_,
                            [b[0] if b[0] != -np.inf else -1e6 for b in bounds],
                            [b[1] if b[1] !=  np.inf else  1e6 for b in bounds])

            lam, rho = self.alpha, self.l1_ratio

            def objective(params):
                coef, intercept = params[:-1], params[-1]
                resid = yn - (Xs @ coef + intercept)
                mse   = np.mean(resid**2)
                l1    = lam * rho       * np.sum(np.abs(coef))
                l2    = lam*(1-rho)/2   * np.sum(coef**2)
                return mse + l1 + l2

            x0 = np.append(coef0, net0.intercept_)
            res = minimize(objective, x0, method='L-BFGS-B',
                           bounds=bounds + [(None, None)],
                           options={'maxiter': 500, 'ftol': 1e-9})
            self.coef_      = res.x[:-1]
            self.intercept_ = res.x[-1]

        yhat = Xs @ self.coef_ + self.intercept_
        self.resid_std_ = float(np.std(yn - yhat))
        return self

    def predict(self, X):
        Xn = X[self.feature_cols_].fillna(0).values.astype(float)
        Xs = self.scaler.transform(Xn)
        return Xs @ self.coef_ + self.intercept_

    def coef_table(self):
        """Return a DataFrame of feature coefficients sorted by |coef|."""
        # Convert scaled coef back to original units
        coef_orig = self.coef_ / self.scaler.scale_
        return (pd.DataFrame({
                    'feature': self.feature_cols_,
                    'coef_orig': coef_orig,
                    'coef_scaled': self.coef_,
                    'bound_lo': [COEF_BOUNDS.get(c,(None,None))[0]
                                 for c in self.feature_cols_],
                    'bound_hi': [COEF_BOUNDS.get(c,(None,None))[1]
                                 for c in self.feature_cols_],
                })
                .assign(abs_coef=lambda d: d['coef_orig'].abs())
```

```
                .sort_values('abs_coef', ascending=False)
                .reset_index(drop=True))


# - Fit both models and compare coefficients ---------------
print("Fitting Elastic Net models on SHORT-TERM features...")
print()

X_train = df.loc[df['split']=='train', FEATURES_SHORT].fillna(0)
y_train = df.loc[df['split']=='train', 'spot_price']

print("  Fitting unconstrained model...")
m_enet_unc = ElasticNetModel(mode='unconstrained').fit(X_train, y_train)

print("  Fitting constrained model (L-BFGS-B)...")
m_enet_con = ElasticNetModel(mode='constrained').fit(X_train, y_train)

# Show top coefficients for both models
ct_unc = m_enet_unc.coef_table().head(12)
ct_con = m_enet_con.coef_table().head(12)

print()
print("Top 12 features - UNCONSTRAINED ElasticNet:")
print(ct_unc[['feature','coef_orig','bound_lo','bound_hi']].to_string(index=False))
print()
print("Top 12 features - CONSTRAINED ElasticNet:")
print(ct_con[['feature','coef_orig','bound_lo','bound_hi']].to_string(index=False))

# Check for constraint violations in unconstrained model
print()
print("Constraint violations in UNCONSTRAINED model:")
n_violations = 0
for _, row in ct_unc.iterrows():
    lo = row['bound_lo']
    hi = row['bound_hi']
    coef = row['coef_orig']
    if lo is not None and coef < lo:
        print(f"  {row['feature']:<30}: coef={coef:.4f} < bound_lo={lo}")
        n_violations += 1
    if hi is not None and coef > hi:
        print(f"  {row['feature']:<30}: coef={coef:.4f} > bound_hi={hi}")
        n_violations += 1
if n_violations == 0:
    print("  None in top-12 features (may exist in lower-ranked features)")
print()
print("The constrained model enforces all bounds by construction.")

def rolling_evaluate(df, model_name, model_factory,
                     horizons, feature_cols=None,
                     window_days=365*3, roll_step=30,
                     verbose=True):
    """
    Expanding-Window Forecast Evaluation Engine

    Design:
```

```
    - Evaluation period: VAL_START to TEST_END (2022-2024)
    - Training window: last 3 years (1095 days) of data before forecast date
    - Window expands by roll_step (30 days) at each step
    - For each evaluation date T and each horizon h:
        1. Train on data from [T - window_days, T]
        2. Predict price at T + h
        3. Compare with actual price at T + h

    Parameters
    ----------
    df           : full DataFrame with features and spot_price
    model_name   : string label ('Naive', 'SARIMA', etc.)
    model_factory: callable that returns a fresh unfitted model instance
    horizons     : list of integer forecast horizons (days)
    feature_cols : feature columns for linear models (None for time-series)
    window_days  : training window length in days (default 3 years)
    roll_step    : re-estimation frequency in days (default 30)
    verbose      : print progress

    Returns
    -------
    DataFrame with columns: forecast_date, target_date, horizon,
                            y_true, y_pred, error, abs_error, sq_error,
                            year, regime, split, model
    """
    df  = df.sort_values('date').reset_index(drop=True)
    # Evaluation dates: VAL_START to TEST_END, stepped by roll_step
    eval_mask  = ((df['date'] >= VAL_START) & (df['date'] <= TEST_END))
    eval_dates = df.loc[eval_mask, 'date'].values[::roll_step]
    n_steps    = len(eval_dates)

    records = []
    t_start = time.time()

    for step_i, fc_ts in enumerate(eval_dates):
        fc_date = pd.Timestamp(fc_ts)

        if verbose and step_i % max(1, n_steps//8) == 0:
            elapsed = time.time() - t_start
            pct     = (step_i+1) / n_steps * 100
            print(f"    [{model_name}] {step_i+1}/{n_steps} "
                  f"({pct:.0f}%)  {fc_date.date()}  {elapsed:.0f}s", end='\r')

        # Training window
        t_start_win = fc_date - pd.Timedelta(days=window_days)
        train_mask  = (df['date'] > t_start_win) & (df['date'] <= fc_date)
        train_df    = df[train_mask].copy()
        if len(train_df) < 90:
            continue

        # Fit model
        try:
            model = model_factory()
            if feature_cols is not None:   # linear models
                X_tr = train_df[feature_cols].fillna(0)
```

```
                y_tr = train_df['spot_price']
                model.fit(X_tr, y_tr)
            else:                           # time-series models
                model.fit(train_df)
        except Exception as e:
            if verbose:
                print(f"\n    [WARN] fit failed at {fc_date.date()}: {e}")
            continue

        # Predict each horizon
        for h in horizons:
            tgt_date  = fc_date + pd.Timedelta(days=h)
            tgt_mask  = df['date'] == tgt_date
            if not tgt_mask.any():
                continue
            y_true = float(df.loc[tgt_mask, 'spot_price'].iloc[0])

            try:
                if feature_cols is not None:
                    # Use FORECAST DATE features (no lookahead)
                    feat_row = df.loc[df['date'] == fc_date, feature_cols]
                    if len(feat_row) == 0:
                        continue
                    y_pred = float(model.predict(feat_row)[0])
                elif model_name == 'Naive':
                    y_pred = model.predict(h, fc_date)
                elif model_name == 'Seasonal-WD':
                    y_pred = model.predict(h, fc_date)
                elif model_name == 'SARIMA':
                    y_pred = model.predict(h)
                else:
                    continue
            except Exception:
                continue

            meta = df.loc[tgt_mask].iloc[0]
            records.append({
                'forecast_date': fc_date,
                'target_date':   tgt_date,
                'horizon':       h,
                'y_true':        y_true,
                'y_pred':        y_pred,
                'error':         y_pred - y_true,
                'abs_error':     abs(y_pred - y_true),
                'sq_error':      (y_pred - y_true)**2,
                'year':          tgt_date.year,
                'regime':        meta.get('regime', ''),
                'split':         meta.get('split',  ''),
                'model':         model_name,
            })

    if verbose:
        print()   # end carriage-return line
    return pd.DataFrame(records)

```

```
700
701 print("Rolling evaluation engine defined.")
702 print()
703
704 # Run All Models
705 # - Evaluation horizons -------------------------
706 # Full set: [1, 7, 14, 30, 60, 90, 180, 360]
707 EVAL_HORIZONS = [1, 7, 14, 30, 90, 180, 360]
708
709 # Feature set selection by horizon (used for linear models)
710 def get_features_for_horizon(h):
711     if h <= 30:
712         return FEATURES_SHORT
713     elif h <= 180:
714         return FEATURES_MEDIUM
715     else:
716         return FEATURES_LONG
717
718 print("="*55)
719 print("RUNNING ROLLING-WINDOW EVALUATION")
720 print(f"Horizons: {EVAL_HORIZONS}")
721 print("="*55)
722 print()
723 print("Model run order:")
724 print("  1. Naive             ")
725 print("  2. Seasonal-WD       ")
726 print("  3. SARIMA            ")
727 print("  4. ElasticNet-Unc    ")
728 print("  5. ElasticNet-Con    ")
729 print()
730 print()
731
732 all_results = []
733
734 # - 1. Naive ------------------------------
735 print("Running Naive...")
736 res_naive = rolling_evaluate(
737     df, 'Naive', NaiveModel,
738     horizons=EVAL_HORIZONS, feature_cols=None, verbose=True
739 )
740 all_results.append(res_naive)
741 print(f"  Done: {len(res_naive)} records")
742
743 # - 2. Seasonal-WD ----------------------------
744 print("Running Seasonal-WD...")
745 res_wd = rolling_evaluate(
746     df, 'Seasonal-WD', SeasonalWDModel,
747     horizons=EVAL_HORIZONS, feature_cols=None, verbose=True
748 )
749 all_results.append(res_wd)
750 print(f"  Done: {len(res_wd)} records")
751
752 # - 3. SARIMA - --------------
753 print("Running SARIMA (this takes a few minutes)...")
754 res_sarima = rolling_evaluate(
```

```
    df, 'SARIMA',
    lambda: SARIMAModel(order=(2,1,1), seasonal_order=(1,0,1,7)),
    horizons=EVAL_HORIZONS, feature_cols=None, verbose=True
)
all_results.append(res_sarima)
print(f"  Done: {len(res_sarima)} records")

# - 4. ElasticNet Unconstrained ----------------------
print("Running ElasticNet-Unconstrained...")
res_unc = rolling_evaluate(
    df, 'ElasticNet-Unconstrained',
    lambda: ElasticNetModel(alpha=0.5, l1_ratio=0.5, mode='unconstrained'),
    horizons=EVAL_HORIZONS,
    feature_cols=FEATURES_SHORT,
    verbose=True
)
all_results.append(res_unc)
print(f"  Done: {len(res_unc)} records")

# - 5. ElasticNet Constrained ------------------------
print("Running ElasticNet-Constrained...")
res_con = rolling_evaluate(
    df, 'ElasticNet-Constrained',
    lambda: ElasticNetModel(alpha=0.5, l1_ratio=0.5, mode='constrained'),
    horizons=EVAL_HORIZONS,
    feature_cols=FEATURES_SHORT,
    verbose=True
)
all_results.append(res_con)
print(f"  Done: {len(res_con)} records")

# - Combine all results --------------------------
results = pd.concat(all_results, ignore_index=True)
results['forecast_date'] = pd.to_datetime(results['forecast_date'])
results['target_date']   = pd.to_datetime(results['target_date'])

print()
print(f"Total evaluation records: {len(results)}")
print(f"Models evaluated: {results['model'].unique().tolist()}")
print(f"Horizons covered: {sorted(results['horizon'].unique().tolist())}")

results.to_csv('results_baseline_NO1.csv', index=False)
print("\nResults saved: results_baseline_NO1.csv")

# Results: RMSE and MAE Tables
def summary_table(results, metric='rmse'):
    """
    Build RMSE or MAE table: rows=models, columns=horizons.
    Mirrors Table 5 in Ghelasi & Ziel (2025).
    """
    fn = (lambda g: np.sqrt(g['sq_error'].mean())) if metric == 'rmse' \
         else (lambda g: g['abs_error'].mean())

    models   = list(results['model'].unique())
    horizons = sorted(results['horizon'].unique())
```

```
    rows     = []
    for model in models:
        sub = results[results['model'] == model]
        row = {'Model': model}
        for h in horizons:
            h_sub = sub[sub['horizon'] == h]
            row[f'h={h}d'] = round(fn(h_sub), 2) if len(h_sub) > 0 else np.nan
        rows.append(row)
    return pd.DataFrame(rows).set_index('Model')


rmse_table = summary_table(results, 'rmse')
mae_table  = summary_table(results, 'mae')

print("="*70)
print("TABLE - RMSE by Model and Horizon (EUR/MWh)")
print("="*70)
print(rmse_table.to_string())
print()
print("="*70)
print("TABLE - MAE by Model and Horizon (EUR/MWh)")
print("="*70)
print(mae_table.to_string())

rmse_table.to_csv('summary_rmse_NO1.csv')
mae_table.to_csv('summary_mae_NO1.csv')
print()
print("Tables saved: summary_rmse_NO1.csv / summary_mae_NO1.csv")

# Figure: Rolling RMSE Over Time
h_plot = 1   # show h=1 day rolling performance over time

fig, ax = plt.subplots(figsize=(15, 6))
fig.suptitle(f'Rolling 30-Day RMSE Over Time  (h={h_plot} day ahead)\n'
             'Red Shaded area = 2022 Crisis (stress-test validation period)',
             fontsize=13, fontweight='bold')

h1_data = results[results['horizon'] == h_plot].copy()

for model in results['model'].unique():
    sub = (h1_data[h1_data['model'] == model]
           .sort_values('target_date'))
    rolling_rmse = (sub['sq_error']
                    .rolling(30, min_periods=10)
                    .mean()
                    .pipe(np.sqrt))
    ax.plot(sub['target_date'], rolling_rmse,
            lw=1.8, alpha=0.9,
            color=MODEL_COLORS.get(model, 'black'),
            label=model)

ax.axvspan(VAL_START, VAL_END, alpha=0.12, color='red', label='Crisis 2022')
ax.axvline(TEST_START, color='green', ls='--', lw=1.2, alpha=0.7,
           label='Test period start')
```

```
ax.set_ylabel('30-Day Rolling RMSE (EUR/MWh)')
ax.set_xlabel('Date')
ax.legend(fontsize=9, ncol=3, loc='upper right')

plt.tight_layout()
plt.show()

# Figure: RMSE and MAE by Horizon
fig, axes = plt.subplots(1, 2, figsize=(16, 7))
fig.suptitle('Forecast Accuracy by Horizon',
             fontsize=13, fontweight='bold')

horizons = sorted(results['horizon'].unique())

for ax, metric, ylabel in [
    (axes[0], 'rmse', 'RMSE (EUR/MWh)'),
    (axes[1], 'mae',  'MAE (EUR/MWh)'),
]:
    fn = (lambda g: np.sqrt(g['sq_error'].mean())) if metric == 'rmse' \
         else (lambda g: g['abs_error'].mean())

    for model in results['model'].unique():
        sub  = results[results['model'] == model]
        vals = [fn(sub[sub['horizon']==h]) if (sub['horizon']==h).any()
                else np.nan for h in horizons]
        ax.plot(horizons, vals,
                marker=MODEL_MARKERS.get(model, 'o'),
                lw=2.0, ms=8,
                color=MODEL_COLORS.get(model, 'k'),
                label=model)

    # Shade the three horizon zones
    ax.axvspan(0,   30,  alpha=0.06, color='steelblue')
    ax.axvspan(30,  180, alpha=0.06, color='orange')
    ax.axvspan(180, 370, alpha=0.06, color='green')
    ax.axvline(30,  color='gray', ls=':', lw=1.2)
    ax.axvline(180, color='gray', ls=':', lw=1.2)

    ymax = ax.get_ylim()[1]
    ax.text(15,  ymax*0.97, 'Short',  ha='center', fontsize=9,
            color='steelblue', fontweight='bold')
    ax.text(100, ymax*0.97, 'Medium', ha='center', fontsize=9,
            color='darkorange', fontweight='bold')
    ax.text(270, ymax*0.97, 'Long',   ha='center', fontsize=9,
            color='darkgreen', fontweight='bold')

    ax.set_xlabel('Forecasting Horizon (days)')
    ax.set_ylabel(ylabel)
    ax.set_xticks(horizons)
    ax.set_xticklabels([f'{h}d' for h in horizons])
    ax.legend(fontsize=9)

plt.tight_layout()
plt.savefig('fig_bm2_rmse_by_horizon.png')
plt.show()
```

```
920
921 # Figure: Annual RMSE Breakdown
922 horizons_to_show = [h for h in [1, 7, 30, 90] if h in results['horizon'].unique()]
923 models = list(results['model'].unique())
924 years = sorted(results['year'].unique())
925
926 fig, axes = plt.subplots(2, 2, figsize=(12, 10), sharey=False)
927 axes = axes.flatten()  # makes indexing easy
928
929 fig.suptitle(
930     'Annual RMSE Breakdown by Horizon\n'
931     '(Red shading = crisis year 2022 - the stress-test validation)',
932     fontsize=13,
933     fontweight='bold'
934 )
935
936 x_pos = np.arange(len(years))
937 width = 0.75 / len(models)
938
939 for ax, h in zip(axes, horizons_to_show):
940     h_df = results[results['horizon'] == h]
941
942     for i, model in enumerate(models):
943         vals = []
944         for yr in years:
945             sub = h_df[(h_df['model'] == model) & (h_df['year'] == yr)]
946             vals.append(np.sqrt(sub['sq_error'].mean()) if len(sub) > 0 else
947             np.nan)
948
949         offset = (i - len(models)/2 + 0.5) * width
950         ax.bar(
951             x_pos + offset,
952             vals,
953             width,
954             color=MODEL_COLORS.get(model, 'gray'),
955             alpha=0.82,
956             label=model if h == horizons_to_show[0] else ''
957         )
958
959     ax.set_xticks(x_pos)
960     ax.set_xticklabels(years, rotation=45, fontsize=8)
961     ax.set_title(f'h = {h} days', fontsize=11)
962     ax.set_ylabel('RMSE (EUR/MWh)')
963
964     # Highlight 2022 crisis year
965     if 2022 in years:
966         idx_2022 = years.index(2022)
967         ax.axvspan(
968             idx_2022 - 0.45,
969             idx_2022 + 0.45,
970             alpha=0.12,
971             color='red',
972             zorder=0
973         )
974
```

```
# Hide any unused axes if fewer than 4 horizons
for ax in axes[len(horizons_to_show):]:
    ax.set_visible(False)

handles = [
    Patch(color=MODEL_COLORS.get(m, 'gray'), alpha=0.82, label=m)
    for m in models
]

fig.legend(
    handles=handles,
    loc='lower center',
    ncol=min(len(models), 4),
    fontsize=9,
    bbox_to_anchor=(0.5, 0.01)
)

plt.tight_layout(rect=[0, 0.06, 1, 0.95])
plt.savefig('fig_bm3_annual_breakdown.png', dpi=300, bbox_inches='tight')
plt.show()

print("Figure BM3 saved: fig_bm3_annual_breakdown.png")
print()
print("Annual RMSE breakdown (RMSE per year):")
for h in horizons_to_show[:3]:
    print(f"\n  Horizon h={h}d:")
    h_df = results[results['horizon'] == h]
    pivot = (
        h_df.groupby(['model', 'year'])['sq_error']
        .apply(lambda x: round(np.sqrt(x.mean()), 1))
        .unstack('year')
    )
    print(pivot.to_string())
print()

# Coefficient stability across horizons ----------------
# Fit the constrained model on the full training window and extract
# coefficients - then compare how they change when using medium vs long
# feature sets.

train_X_s = df.loc[df['split']=='train', FEATURES_SHORT].fillna(0)
train_X_m = df.loc[df['split']=='train', FEATURES_MEDIUM].fillna(0)
train_X_l = df.loc[df['split']=='train', FEATURES_LONG].fillna(0)
train_y   = df.loc[df['split']=='train', 'spot_price']

print("Fitting constrained ElasticNet on all three feature tiers...")
m_short  = ElasticNetModel(mode='constrained').fit(train_X_s, train_y)
m_medium = ElasticNetModel(mode='constrained').fit(train_X_m, train_y)
m_long   = ElasticNetModel(mode='constrained').fit(train_X_l, train_y)

# Key features that appear in multiple tiers
shared_features = list(set(FEATURES_SHORT) &
                       set(FEATURES_MEDIUM) &
                       set(FEATURES_LONG))
interest_feats  = [
```

```
    'forward_Y1', 'epad_Y1', 'hydro_reserve_gwh',
    'fuel_gas_eur_mwh', 'macro_eur_nok',
    'season_winter', 'month_num',
]
interest_feats  = [f for f in interest_feats if f in shared_features]

fig, axes = plt.subplots(2, 1, figsize=(10, 12))

# Left panel: bar chart of shared feature coefficients across tiers
ax = axes[0]
if interest_feats:
    n_f = len(interest_feats)
    x   = np.arange(n_f)
    w   = 0.25

    def get_coef(model, feat):
        ct = model.coef_table()
        row = ct[ct['feature'] == feat]
        return float(row['coef_orig'].iloc[0]) if len(row) > 0 else 0.0

    coefs_s = [get_coef(m_short,  f) for f in interest_feats]
    coefs_m = [get_coef(m_medium, f) for f in interest_feats]
    coefs_l = [get_coef(m_long,   f) for f in interest_feats]

    ax.bar(x - w, coefs_s, w, color=COLORS['short'],  alpha=0.82,
    label='Short tier')
    ax.bar(x,     coefs_m, w, color=COLORS['medium'], alpha=0.82,
    label='Medium tier')
    ax.bar(x + w, coefs_l, w, color=COLORS['long'],   alpha=0.82,
    label='Long tier')
    ax.set_xticks(x)
    ax.set_xticklabels(interest_feats, rotation=45, ha='right', fontsize=9)
    ax.axhline(0, color='k', lw=0.8)
    #ax.set_title('Shared Feature Coefficients Across Tiers\n'
    #             '(physical constraints ensure correct signs)')
    ax.set_ylabel('Original-scale coefficient')
    ax.legend(fontsize=9)

# Right panel: top-10 coefficients for each tier
ax2 = axes[1]
for i, (model, label, color) in enumerate([
    (m_short,  'Short (h<=30d)',    COLORS['short']),
    (m_medium, 'Medium (h=1-6mo)', COLORS['medium']),
    (m_long,   'Long (h=6-12mo)',  COLORS['long']),
]):
    ct = model.coef_table().head(10)
    ct = ct.sort_values('coef_orig')
    y_pos = np.arange(len(ct)) + i * 12
    ax2.barh(y_pos, ct['coef_orig'].values, 0.8,
             color=color, alpha=0.75, label=label)
    for y, feat in zip(y_pos, ct['feature'].values):
        ax2.text(0.02, y, feat, va='center', fontsize=7,
                 color='black', ha='left')
ax2.axvline(0, color='k', lw=0.8)
ax2.set_title('Top 10 Features per Tier\n(most influential coefficients)')
```

```
ax2.set_xlabel('Original-scale coefficient')
ax2.legend(fontsize=9, loc='lower right')
ax2.set_yticks([])

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

print("Figure BM4 saved: fig_bm4_coef_stability.png")
print()
print("Full constrained coefficient table - SHORT tier (top 15):")
print(m_short.coef_table()[['feature','coef_orig','bound_lo','bound_hi']]
      .head(15).to_string(index=False))
print()
print("Physical constraint check:")
ct_full = m_short.coef_table()
for _, row in ct_full.iterrows():
    lo, hi = row['bound_lo'], row['bound_hi']
    c = row['coef_orig']
    if lo is not None and not np.isnan(c) and c < lo - 0.001:
        print(f"  VIOLATION: {row['feature']} coef={c:.4f} < bound={lo}")
    if hi is not None and not np.isnan(c) and c > hi + 0.001:
        print(f"  VIOLATION: {row['feature']} coef={c:.4f} > bound={hi}")

# Residual Diagnostic Tests for All 5 Baseline Models
# ============================================================================
# ROLLING RESIDUAL COLLECTION
# ============================================================================
HORIZONS   = [1, 7, 30, 90]
WINDOW     = 365 * 3   # 3-year rolling window
ROLL_STEP  = 20        # evaluate every 20 days (gives ~37 steps)

test_dates = df.loc[
    (df['date'] >= TEST_START) & (df['date'] <= TEST_END),
    'date'
].values[::ROLL_STEP]

MODELS = [
    'Naive',
    'Seasonal-WD',
    'SARIMA',
    'ElasticNet-Unconstrained',
    'ElasticNet-Constrained',
]

all_resid = {m: {h: [] for h in HORIZONS} for m in MODELS}

def season_label(m):
    return {12:'W', 1:'W', 2:'W',
            3:'Sp', 4:'Sp', 5:'Sp',
            6:'Su', 7:'Su', 8:'Su',
            9:'A', 10:'A', 11:'A'}[m]

print(f"\nCollecting residuals over {len(test_dates)} rolling evaluation steps")
print("This may take 5-10 minutes due to SARIMA fitting at each step.\n")
```

```
for step_i, fc_ts in enumerate(test_dates):
    fc = pd.Timestamp(fc_ts)
    tw = fc - pd.Timedelta(days=WINDOW)
    tr = df[(df['date'] > tw) & (df['date'] <= fc)].copy()
    if len(tr) < 90:
        continue

    print(f"  Step {step_i+1}/{len(test_dates)}   |   {fc.date()}", end='\r')

    # - Weekday / season lookups ---------------------
    wd_means = tr.groupby(tr['date'].dt.dayofweek)['spot_price'].mean().to_dict()
    last_p   = float(tr['spot_price'].iloc[-1])
    tr2      = tr.assign(
        dow_=tr['date'].dt.dayofweek,
        sea_=tr['date'].dt.month.map(season_label)
    )
    wd_sea   = tr2.groupby(['dow_', 'sea_'])['spot_price'].mean().to_dict()

    # - Fit ElasticNet-Unconstrained -------------------
    try:
        sc_u  = StandardScaler()
        Xs_u  = sc_u.fit_transform(tr[FEATURES].fillna(0).values.astype(float))
        net_u = ElasticNet(alpha=0.5, l1_ratio=0.5,
                           max_iter=1000, fit_intercept=True)
        net_u.fit(Xs_u, tr['spot_price'].values.astype(float))
        en_u_ok = True
    except Exception:
        en_u_ok = False

    # - Fit ElasticNet-Constrained --------------------
    # Uses same fit as unconstrained then clips coefficients to enforce
    # physical sign constraints from the Norwegian hydro merit-order
    try:
        sc_c  = StandardScaler()
        Xs_c  = sc_c.fit_transform(tr[FEATURES].fillna(0).values.astype(float))
        net_c = ElasticNet(alpha=0.5, l1_ratio=0.5,
                           max_iter=1000, fit_intercept=True)
        net_c.fit(Xs_c, tr['spot_price'].values.astype(float))
        for j, col in enumerate(FEATURES):
            if col in COEF_BOUNDS_SIGN:
                lo, hi = COEF_BOUNDS_SIGN[col]
                c = net_c.coef_[j]
                net_c.coef_[j] = np.clip(c, lo * abs(c), hi * abs(c))
        en_c_ok = True
    except Exception:
        en_c_ok = False

    # - Fit SARIMA(2,1,1)(1,0,1)[7] -------------------
    sarima_ok        = False
    sarima_forecasts = {}
    try:
        y_s  = tr['spot_price'].values.astype(float)
        mdl  = SARIMAX(y_s, order=(2, 1, 1),
                       seasonal_order=(1, 0, 1, 7),
                       enforce_stationarity=False,
```

```
                        enforce_invertibility=False)
            res_s = mdl.fit(disp=False, maxiter=50)
            fc_s  = res_s.forecast(steps=max(HORIZONS))
            for h in HORIZONS:
                sarima_forecasts[h] = float(fc_s[h - 1])
            sarima_ok = True
        except Exception:
            pass

        # - Collect residuals for each horizon ----------------
        for h in HORIZONS:
            tgt  = fc + pd.Timedelta(days=h)
            mask = df['date'] == tgt
            if not mask.any():
                continue
            y_true = float(df.loc[mask, 'spot_price'].values[0])

            # Naive
            y_naive = last_p if h <= 7 else float(wd_means.get(tgt.dayofweek, last_p))
            all_resid['Naive'][h].append(y_true - y_naive)

            # Seasonal-WD
            y_wd = float(wd_sea.get(
                (tgt.dayofweek, season_label(tgt.month)),
                float(tr['spot_price'].mean())
            ))
            all_resid['Seasonal-WD'][h].append(y_true - y_wd)

            # SARIMA
            if sarima_ok:
                all_resid['SARIMA'][h].append(y_true - sarima_forecasts[h])

            # ElasticNet-Unconstrained
            if en_u_ok:
                fr = df.loc[df['date'] == fc, FEATURES].fillna(0).values.astype(float)
                if len(fr) > 0:
                    y_en_u = float(net_u.predict(sc_u.transform(fr))[0])
                    all_resid['ElasticNet-Unconstrained'][h].append(y_true - y_en_u)

            # ElasticNet-Constrained
            if en_c_ok:
                fr = df.loc[df['date'] == fc, FEATURES].fillna(0).values.astype(float)
                if len(fr) > 0:
                    y_en_c = float(net_c.predict(sc_c.transform(fr))[0])
                    all_resid['ElasticNet-Constrained'][h].append(y_true - y_en_c)

print("\n\nResidual collection complete.")
print(f"{'Model':<26}  " + "  ".join([f"h={h}" for h in HORIZONS]))
print("-" * 55)
for m in MODELS:
    counts = [len(all_resid[m][h]) for h in HORIZONS]
    print(f"{m:<26}  " + "  ".join([f"{c:>4}" for c in counts]))

# ============================================================================
# RUN DIAGNOSTIC TESTS
```

```
# ============================================================================

def run_diagnostics(e_list):
    """
    Run Ljung-Box, ARCH-LM and Jarque-Bera on a residual list.
    Returns a dict of test statistics and p-values.
    """
    e = np.array(e_list, dtype=float)
    if len(e) < 8:
        return None

    # Ljung-Box (10 lags) - tests for residual autocorrelation
    lb     = acorr_ljungbox(e, lags=[10], return_df=True)
    lb_s   = round(float(lb['lb_stat'].iloc[0]),   2)
    lb_p   = round(float(lb['lb_pvalue'].iloc[0]), 4)

    # ARCH-LM (5 lags) - tests for conditional heteroskedasticity
    try:
        r       = het_arch(e, nlags=5)
        arch_s  = round(float(r[0]), 2)
        arch_p  = round(float(r[1]), 4)
    except Exception:
        arch_s, arch_p = np.nan, np.nan

    # Jarque-Bera - tests for normality
    jb_s, jb_p, skew, kurt = jarque_bera(e)

    return {
        'lb_stat':   lb_s,
        'lb_p':      lb_p,
        'arch_stat': arch_s,
        'arch_p':    arch_p,
        'jb_stat':   round(float(jb_s),   2),
        'jb_p':      round(float(jb_p),   4),
        'skew':      round(float(skew),   3),
        'kurt':      round(float(kurt),   3),
        'rmse':      round(float(np.sqrt(np.mean(e**2))), 2),
        'bias':      round(float(np.mean(e)),             2),
        'n':         int(len(e)),
    }


rows = []
for model in MODELS:
    for h in HORIZONS:
        d = run_diagnostics(all_resid[model][h])
        if d:
            d['model']   = model
            d['horizon'] = h
            rows.append(d)

diag_df = pd.DataFrame(rows)
diag_df.to_csv('residual_diagnostics_NO1.csv', index=False)

# ============================================================================
```

```
# PRINT RESULTS TABLE
# ============================================================================

def sig(p):
    """Return significance stars."""
    if np.isnan(p): return '   '
    if p < 0.01:    return '** '
    if p < 0.05:    return '*  '
    return '   '

print()
print("=" * 100)
print("RESIDUAL DIAGNOSTIC TESTS - ALL 5 BASELINE MODELS")
print("Evaluation period: Jan 2023 - Dec 2024 (test set)")
print("* p < 0.05   ** p < 0.01")
print()
print("LB   = Ljung-Box portmanteau test (lags=10)  - H0: no residual
autocorrelation")
print("ARCH = ARCH-LM test (lags=5)                 - H0: no conditional
heteroskedasticity")
print("JB   = Jarque-Bera test                      - H0: residuals are normally
distributed")
print("=" * 100)

header = (f"{'Model':<26} {'h':>4} {'n':>4}  "
          f"{'LB stat':>8} {'LB p':>8}  "
          f"{'ARCH stat':>10} {'ARCH p':>8}  "
          f"{'JB stat':>8} {'JB p':>8}  "
          f"{'Skew':>7} {'Kurt':>7}  {'RMSE':>8}")
print(header)
print("-" * 100)

prev_model = None
for _, row in diag_df.sort_values(['model', 'horizon']).iterrows():
    if prev_model and row['model'] != prev_model:
        print()
    prev_model = row['model']
    print(
        f"{row['model']:<26} {int(row['horizon']):>4} {int(row['n']):>4}  "
        f"{row['lb_stat']:>8.2f} {row['lb_p']:>7.4f}{sig(row['lb_p'])} "
        f"{row['arch_stat']:>10.2f} {row['arch_p']:>7.4f}{sig(row['arch_p'])} "
        f"{row['jb_stat']:>8.2f} {row['jb_p']:>7.4f}{sig(row['jb_p'])} "
        f"{row['skew']:>7.3f} {row['kurt']:>7.3f}  {row['rmse']:>8.2f}"
    )

print()
print("Saved: residual_diagnostics_NO1.csv")

# ============================================================================
# FIGURE: RESIDUAL DIAGNOSTICS AT h=1d FOR ALL 5 MODELS
# ============================================================================

fig, axes = plt.subplots(5, 3, figsize=(17, 21))
fig.suptitle(
    'Residual Diagnostics - All 5 Baseline Models  |  h = 1 day  |  Test Set
```

```
    2023-2024\n'
    'Left: Residual time series  |  Centre: ACF  |  Right: Normal Q-Q plot',
    fontsize=12, fontweight='bold'
)

for i, model in enumerate(MODELS):
    e = np.array(all_resid[model][1], dtype=float)

    if len(e) < 5:
        for j in range(3):
            axes[i, j].text(0.5, 0.5, 'Insufficient data',
                            ha='center', va='center',
                            transform=axes[i, j].transAxes)
            axes[i, j].set_title(model, fontsize=10, fontweight='bold')
        continue

    d = run_diagnostics(e)

    # - Panel 1: Residual time series -----------------
    ax1 = axes[i, 0]
    ax1.plot(e, lw=0.9, color='steelblue', alpha=0.85)
    ax1.axhline(0, color='black', lw=1.2)
    ax1.axhline( 2 * e.std(), color='red', ls='--', lw=1, alpha=0.7, label='±2σ')
    ax1.axhline(-2 * e.std(), color='red', ls='--', lw=1, alpha=0.7)
    ax1.set_title(model, fontsize=10, fontweight='bold')
    ax1.set_ylabel('Residual (EUR/MWh)')
    ax1.set_xlabel('Evaluation step')
    ax1.legend(fontsize=8)

    # Annotate with test statistics
    def star(p): return '**' if p < 0.01 else '*' if p < 0.05 else ''
    ann = (
        f"LB p = {d['lb_p']:.3f}{star(d['lb_p'])}"
        f"   ARCH p = {d['arch_p']:.3f}{star(d['arch_p'])}\n"
        f"JB p = {d['jb_p']:.3f}{star(d['jb_p'])}"
        f"   Skew = {d['skew']:.2f}   Kurt = {d['kurt']:.2f}\n"
        f"RMSE = {d['rmse']:.1f} EUR/MWh   Bias = {d['bias']:.1f}"
    )
    ax1.text(0.02, 0.03, ann, transform=ax1.transAxes, fontsize=8,
             bbox=dict(fc='white', ec='gray', alpha=0.85), va='bottom')

    # - Panel 2: ACF --------------------------
    ax2 = axes[i, 1]
    plot_acf(e, lags=min(15, len(e) // 2 - 1),
             ax=ax2, zero=False, alpha=0.05)
    ax2.set_title('ACF of Residuals', fontsize=9)
    ax2.set_xlabel('Lag (evaluation steps)')

    # - Panel 3: Normal Q-Q plot --------------------
    ax3 = axes[i, 2]
    stats.probplot(e, dist='norm', plot=ax3)
    ax3.set_title('Normal Q-Q Plot', fontsize=9)
    ax3.get_lines()[0].set(markersize=5, alpha=0.65, color='steelblue')
    ax3.get_lines()[1].set(color='red', lw=1.5)

```

```
plt.tight_layout()
plt.show()
```

Appendix F

Source Code for Probabilistic Forecasting

```
import warnings; warnings.filterwarnings('ignore')
import time, math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.patches import Patch
from scipy import stats
from scipy.stats import pearsonr, spearmanr, johnsonsu, norm, kstest
from sklearn.linear_model import ElasticNet
from sklearn.preprocessing import StandardScaler

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

plt.rcParams.update({
    'figure.facecolor': 'white',  'axes.facecolor': '#f8f9fa',
    'axes.grid': True,            'grid.alpha': 0.35,
    'font.size': 11,              'axes.spines.top': False,
    'axes.spines.right': False,   'axes.labelsize': 12,
    'axes.titlesize': 13,         'figure.dpi': 110,
    'savefig.dpi': 150,           'savefig.bbox': 'tight',
})

COLORS = {
    'pre': '#2196F3', 'crisis': '#F44336', 'post': '#4CAF50',
    'hydro': '#00BCD4', 'gas': '#FF9800', 'fwd': '#9C27B0',
}
MODEL_COLORS = {
    'Naive-Gaussian': '#9E9E9E',
    'LASSO-QR':       '#2196F3',
    'DMLP':           '#4CAF50',
}

CRISIS_START = pd.Timestamp('2021-07-01')
CRISIS_END   = pd.Timestamp('2023-01-01')
TRAIN_END    = pd.Timestamp('2021-12-31')
```

```
VAL_START    = pd.Timestamp('2022-01-01')
VAL_END      = pd.Timestamp('2022-12-31')
TEST_START   = pd.Timestamp('2023-01-01')
TEST_END     = pd.Timestamp('2024-12-31')

# - Quantile levels ---------------------------
# TAUS = [round(q/100, 2) for q in range(1, 100)]  # 99 levels - production

COVERAGE_LEVELS = [0.50, 0.80, 0.90]
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
MONTHS = ['Jan','Feb','Mar','Apr','May','Jun',
          'Jul','Aug','Sep','Oct','Nov','Dec']

# Evaluation Metrics (CRPS, PIT, Coverage)
def pinball_loss(y_true, y_pred_q, tau):
    e = np.array(y_true) - np.array(y_pred_q)
    return float(np.mean(np.where(e >= 0, tau * e, (tau - 1) * e)))

def crps_from_quantiles(y_true, q_matrix, taus):
    total = sum(pinball_loss(y_true, q_matrix[:, j], tau)
                for j, tau in enumerate(taus))
    return 2.0 * total / len(taus)

def compute_pit(y_true, q_matrix, taus):
    taus_arr = np.array(taus)
    pits = []
    for i, y in enumerate(y_true):
        q_row = q_matrix[i]
        if y <= q_row[0]:      pits.append(float(taus_arr[0]))
        elif y >= q_row[-1]:   pits.append(float(taus_arr[-1]))
        else:
            idx = np.searchsorted(q_row, y, side='left')
            t0, t1 = taus_arr[idx-1], taus_arr[idx]
            q0, q1 = q_row[idx-1], q_row[idx]
            pits.append(float(t0 + (t1 - t0) * (y - q0) / (q1 - q0 + 1e-9)))
    return np.array(pits)

print("Metrics defined: pinball_loss, crps_from_quantiles, compute_pit")

# Model 1: Naive Gaussian Benchmark
class NaiveGaussian:
    """
    Naive Gaussian Benchmark.
    Point forecast: weekday-corrected random walk.
    Uncertainty: Gaussian with std = sigma_residual * sqrt(h).
    """
    name = 'Naive-Gaussian'

    def fit(self, train_df):
        self.last_price    = float(train_df['spot_price'].iloc[-1])
        self.weekday_means = (train_df
                              .groupby(train_df['date'].dt.dayofweek)['spot_price']
                              .mean().to_dict())
        resids             = train_df['spot_price'] - train_df['spot_price']
        .shift(1)
```

```
        self.resid_std     = float(resids.dropna().std())
        return self

    def _point(self, horizon, forecast_date):
        if horizon <= 7:
            return self.last_price
        dow = (forecast_date + pd.Timedelta(days=horizon)).dayofweek
        return self.weekday_means.get(dow, self.last_price)

    def predict_quantiles(self, horizon, forecast_date, taus):
        mu  = self._point(horizon, forecast_date)
        std = self.resid_std * math.sqrt(horizon)
        return np.array([norm.ppf(tau, loc=mu, scale=std) for tau in taus])

print("NaiveGaussian defined.")

# Model 2: LASSO Quantile Regression
class LASSOQuantileRegression:
    """
    LASSO Quantile Regression - (Agakishiev et al. 2025), Section 3.4.
    Estimates 99 quantile levels using L1-regularised pinball loss.
    lambda = 0.01 (fixed).
    """
    name = 'LASSO-QR'

    def __init__(self, lam=0.01, taus=None):
        self.lam    = lam
        self.taus   = taus if taus is not None else TAUS
        self.models = {}
        self.scaler = StandardScaler()
        self.feature_cols_ = []

    def fit(self, X, y):
        self.feature_cols_ = list(X.columns)
        Xn = X.fillna(0).values.astype(float)
        yn = y.fillna(float(y.median())).values.astype(float)
        Xs = self.scaler.fit_transform(Xn)
        for tau in self.taus:
            n   = len(yn)
            wt  = np.ones(n)
            coef, intercept = np.zeros(Xs.shape[1]), float(np.mean(yn))
            for _ in range(3):
                resid = yn - (Xs @ coef + intercept)
                wt    = np.where(resid >= 0, tau, 1 - tau)
                wt    = np.clip(wt, 0.01, None)
                net   = ElasticNet(alpha=self.lam, l1_ratio=1.0, max_iter=1000,
                                   fit_intercept=True, warm_start=True)
                net.fit(Xs, yn, sample_weight=wt)
                coef, intercept = net.coef_, net.intercept_
            self.models[tau] = (coef.copy(), float(intercept))
        return self

    def predict_quantiles(self, X):
        Xn = X[self.feature_cols_].fillna(0).values.astype(float)
        Xs = self.scaler.transform(Xn)
```

```
        q  = np.zeros((Xs.shape[0], len(self.taus)))
        for j, tau in enumerate(self.taus):
            coef, intercept = self.models[tau]
            q[:, j] = Xs @ coef + intercept
        return np.sort(q, axis=1)

print("LASSOQuantileRegression defined.")

# Model 3: DMLP with Johnson's SU Distribution
class JohnsonSULayer(nn.Module):
    def __init__(self, in_features, eps=1e-3):
        super().__init__()
        self.fc  = nn.Linear(in_features, 4)
        self.eps = eps

    def forward(self, x):
        raw = self.fc(x)
        g   = raw[:, 0]
        d   = nn.functional.softplus(raw[:, 1]) + self.eps
        xi  = raw[:, 2]
        lam = nn.functional.softplus(raw[:, 3]) + self.eps
        return g, d, xi, lam


class DMLPNetwork(nn.Module):
    """
    Architecture (Paper 1, Agakishiev et al. 2025, Section 3.3):
        Input -> [100 -> 50 -> 25] (BatchNorm + ReLU + Dropout 0.3)
              -> Johnson SU layer (gamma, delta, xi, lambda)
    Loss: Negative Log-Likelihood of the Johnson SU distribution.
    """
    def __init__(self, n_features, dropout=0.3):
        super().__init__()
        self.hidden = nn.Sequential(
            nn.Linear(n_features, 100), nn.BatchNorm1d(100), nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(100, 50),         nn.BatchNorm1d(50),  nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(50, 25),          nn.BatchNorm1d(25),  nn.ReLU(),
            nn.Dropout(dropout),
        )
        self.dist = JohnsonSULayer(25)

    def forward(self, x):
        return self.dist(self.hidden(x))

    def nll_loss(self, x, y):
        g, d, xi, lam = self.forward(x)
        d   = d.clamp(min=1e-3)
        lam = lam.clamp(min=1e-3)
        z   = g + d * torch.asinh((y - xi) / lam)
        log_pdf = (torch.log(d) - torch.log(lam)
                   - 0.5 * torch.log(1 + ((y - xi) / lam) ** 2)
                   - 0.5 * np.log(2 * np.pi) - 0.5 * z ** 2)
        return -log_pdf.mean()
```

```
205
206     def get_params(self, x):
207         with torch.no_grad():
208             g, d, xi, lam = self.forward(x)
209         return (g.cpu().numpy(), d.cpu().numpy(),
210                 xi.cpu().numpy(), lam.cpu().numpy())
211
212
213 def jsu_quantile(tau, g, d, xi, lam):
214     return xi + lam * np.sinh((norm.ppf(tau) - g) / d)
215
216
217 class DMLPModel:
218     """
219     Training wrapper for DMLPNetwork.
220     Adam optimiser, lr=1e-3, weight_decay=1e-4, early stopping patience=15.
221     """
222     name = 'DMLP'
223
224     def __init__(self, epochs=150, batch_size=256, lr=1e-3,
225                  weight_decay=1e-4, patience=15, dropout=0.3, taus=None):
226         self.epochs       = epochs
227         self.batch_size   = batch_size
228         self.lr           = lr
229         self.weight_decay = weight_decay
230         self.patience     = patience
231         self.dropout      = dropout
232         self.taus         = taus if taus is not None else TAUS
233         self.scaler       = StandardScaler()
234         self.net_         = None
235         self.feature_cols_= []
236         self.train_losses = []
237         self.val_losses   = []
238
239     def fit(self, X, y):
240         self.feature_cols_ = list(X.columns)
241         Xn = X.fillna(0).values.astype(float)
242         yn = y.fillna(float(y.median())).values.astype(float)
243         Xs = self.scaler.fit_transform(Xn)
244
245         n_val   = max(30, int(0.15 * len(Xs)))
246         Xt, Xv  = Xs[:-n_val], Xs[-n_val:]
247         yt, yv  = yn[:-n_val], yn[-n_val:]
248
249         Xt_t = torch.tensor(Xt, dtype=torch.float32).to(DEVICE)
250         yt_t = torch.tensor(yt, dtype=torch.float32).to(DEVICE)
251         Xv_t = torch.tensor(Xv, dtype=torch.float32).to(DEVICE)
252         yv_t = torch.tensor(yv, dtype=torch.float32).to(DEVICE)
253
254         loader = DataLoader(TensorDataset(Xt_t, yt_t),
255                             batch_size=self.batch_size, shuffle=True)
256         self.net_ = DMLPNetwork(Xs.shape[1], self.dropout).to(DEVICE)
257         opt   = optim.Adam(self.net_.parameters(),
258                            lr=self.lr, weight_decay=self.weight_decay)
259         sched = optim.lr_scheduler.ReduceLROnPlateau(
```

```
            opt, patience=5, factor=0.5, min_lr=1e-5)

        best_val   = np.inf
        best_state = None
        no_imp     = 0
        self.train_losses = []
        self.val_losses   = []

        for epoch in range(self.epochs):
            self.net_.train()
            ep_loss = 0.0
            for Xb, yb in loader:
                opt.zero_grad()
                loss = self.net_.nll_loss(Xb, yb)
                loss.backward()
                nn.utils.clip_grad_norm_(self.net_.parameters(), 1.0)
                opt.step()
                ep_loss += loss.item() * len(Xb)
            ep_loss /= len(Xt_t)

            self.net_.eval()
            with torch.no_grad():
                vl = self.net_.nll_loss(Xv_t, yv_t).item()
            self.train_losses.append(ep_loss)
            self.val_losses.append(vl)
            sched.step(vl)

            if vl < best_val - 1e-5:
                best_val   = vl
                best_state = {k: v.clone()
                              for k, v in self.net_.state_dict().items()}
                no_imp     = 0
            else:
                no_imp += 1
                if no_imp >= self.patience:
                    break

        if best_state:
            self.net_.load_state_dict(best_state)
        return self

    def predict_quantiles(self, X):
        Xn = X[self.feature_cols_].fillna(0).values.astype(float)
        Xs = self.scaler.transform(Xn)
        Xt = torch.tensor(Xs, dtype=torch.float32).to(DEVICE)
        self.net_.eval()
        g_a, d_a, xi_a, lam_a = self.net_.get_params(Xt)
        taus = np.array(self.taus)
        q    = np.zeros((len(g_a), len(taus)))
        for i in range(len(g_a)):
            q[i] = jsu_quantile(taus, float(g_a[i]), float(d_a[i]),
                                float(xi_a[i]), float(lam_a[i]))
        return np.sort(q, axis=1)
```

```python
# Verify architecture
n_feat   = len(FEATURES_SHORT)
net_test = DMLPNetwork(n_feat)
n_params = sum(p.numel() for p in net_test.parameters())
print(f"DMLP defined. Input={n_feat} features, Parameters={n_params:,}")

# Rolling Evaluation Engine with Horizon-Specific Features
def rolling_prob_evaluate(df, model_name, model_factory,
                          horizons, feature_selector,
                          window_days=365*3, roll_step=30,
                          taus=TAUS, verbose=True):
    """
    Rolling-window probabilistic evaluation with HORIZON-SPECIFIC FEATURE SETS.
        h <= 30d    => FEATURES_SHORT  (AR lags + all fundamentals)
        30 < h <= 180d => FEATURES_MEDIUM (no AR lags, seasonal deviations)
        h > 180d    => FEATURES_LONG  (only year-ahead forwards + seasonal)
    """
    all_feat_cols = feature_selector(1)   # FEATURES_SHORT is the broadest set

    df         = df.sort_values('date').reset_index(drop=True)
    eval_mask  = (df['date'] >= VAL_START) & (df['date'] <= TEST_END)
    eval_dates = df.loc[eval_mask, 'date'].values[::roll_step]
    n_steps    = len(eval_dates)
    records    = []
    t0         = time.time()

    for step_i, fc_ts in enumerate(eval_dates):
        fc_date = pd.Timestamp(fc_ts)

        if verbose and step_i % max(1, n_steps // 8) == 0:
            elapsed = time.time() - t0
            print(f"    [{model_name}] {step_i+1}/{n_steps} "
                  f"({100*(step_i+1)/n_steps:.0f}%)  "
                  f"{fc_date.date()}  {elapsed:.0f}s", end='\r')

        t_win    = fc_date - pd.Timedelta(days=window_days)
        train_df = df[(df['date'] > t_win) & (df['date'] <= fc_date)].copy()
        if len(train_df) < 90:
            continue

        # Fit on the SHORT (broadest) feature set
        try:
            model = model_factory()
            if model_name == 'Naive-Gaussian':
                model.fit(train_df)
            else:
                X_tr = train_df[all_feat_cols].fillna(0)
                y_tr = train_df['spot_price']
                model.fit(X_tr, y_tr)
        except Exception as e:
            if verbose:
                print(f"\n    [WARN] fit failed at {fc_date.date()}: {e}")
            continue

        for h in horizons:
```

```
            tgt_date = fc_date + pd.Timedelta(days=h)
            tgt_mask = df['date'] == tgt_date
            if not tgt_mask.any():
                continue
            y_true = float(df.loc[tgt_mask, 'spot_price'].iloc[0])

            # Get horizon-appropriate feature columns
            feat_cols_h = feature_selector(h)
            tier        = 'short' if h <= 30 else 'medium' if h <= 180 else 'long'

            try:
                if model_name == 'Naive-Gaussian':
                    q_vec = model.predict_quantiles(h, fc_date, taus)
                else:
                    # Build a full-width feature row using FORECAST DATE values
                    feat_full = df.loc[df['date'] == fc_date, all_feat_cols]
                    .fillna(0).copy()
                    if len(feat_full) == 0:
                        continue
                    # Zero out features excluded at this horizon
                    # This prevents AR lags from influencing medium/long
                    predictions
                    excluded = set(all_feat_cols) - set(feat_cols_h)
                    for col in excluded:
                        if col in feat_full.columns:
                            feat_full[col] = 0.0
                    q_vec = model.predict_quantiles(feat_full)[0]
            except Exception:
                continue

            q_vec  = np.array(q_vec)
            taus_a = np.array(taus)
            crps_v = crps_from_quantiles(np.array([y_true]),
                                         q_vec.reshape(1, -1), taus)

            def q_at(lv):
                return float(q_vec[np.argmin(np.abs(taus_a - lv))])

            meta = df.loc[tgt_mask].iloc[0]
            records.append({
                'forecast_date': fc_date,
                'target_date':   tgt_date,
                'horizon':       h,
                'feature_tier':  tier,
                'y_true':        y_true,
                'crps':          crps_v,
                'q05': q_at(0.05), 'q10': q_at(0.10), 'q25': q_at(0.25),
                'q50': q_at(0.50), 'q75': q_at(0.75), 'q90': q_at(0.90),
                'q95': q_at(0.95),
                'model':   model_name,
                'year':    tgt_date.year,
                'regime':  meta.get('regime', ''),
                'split':   meta.get('split',  ''),
            })
```

```
    if verbose:
        print()
    return pd.DataFrame(records)


# - Horizons ------------------------------
PROB_HORIZONS = [1, 7, 30, 90, 180, 360]

print("Rolling evaluation engine defined.")
print(f"Evaluation horizons: {PROB_HORIZONS}")
print()
print("Feature tier assigned at each horizon:")
for h in PROB_HORIZONS:
    feat_cols_h = get_features(h)
    tier = 'SHORT' if h <= 30 else 'MEDIUM' if h <= 180 else 'LONG'
    print(f"  h={h:3d}d  =>  FEATURES_{tier}  ({len(feat_cols_h)}
    features active)")

# Run All Models
print("="*60)
print("PROBABILISTIC MODEL EVALUATION - Horizon-Specific Features")
print(f"Horizons : {PROB_HORIZONS}")
print(f"Quantiles: {len(TAUS)} levels")
print("="*60)
print()
print("Estimated runtimes (CPU):")
print("  Naive-Gaussian : ~5s")
print("  LASSO-QR       : ~5-10 min")
print("  DMLP           : ~20-30 min")
print()

prob_results = []

# - 1. Naive Gaussian ---------------------------
print("Running Naive-Gaussian...")
res_ng = rolling_prob_evaluate(
    df, 'Naive-Gaussian', NaiveGaussian,
    horizons=PROB_HORIZONS,
    feature_selector=get_features,
    verbose=True,
)
prob_results.append(res_ng)
print(f"  Done: {len(res_ng)} records")

# - 2. LASSO-QR ------------------------------
print("\nRunning LASSO-QR...")
res_lqr = rolling_prob_evaluate(
    df, 'LASSO-QR',
    lambda: LASSOQuantileRegression(lam=0.01),
    horizons=PROB_HORIZONS,
    feature_selector=get_features,
    verbose=True,
)
prob_results.append(res_lqr)
print(f"  Done: {len(res_lqr)} records")
```

```
480
481 # - 3. DMLP  (comment out this block to skip) ---------------
482 print("\nRunning DMLP (takes ~20-30 min on CPU)...")
483 res_dmlp = rolling_prob_evaluate(
484     df, 'DMLP',
485     lambda: DMLPModel(epochs=150, batch_size=256, lr=1e-3,
486                       weight_decay=1e-4, patience=15, dropout=0.3),
487     horizons=PROB_HORIZONS,
488     feature_selector=get_features,
489     verbose=True,
490 )
491 prob_results.append(res_dmlp)
492 print(f"  Done: {len(res_dmlp)} records")
493
494 # - Combine --------------------------------
495 prob_df = pd.concat(prob_results, ignore_index=True)
496 prob_df['forecast_date'] = pd.to_datetime(prob_df['forecast_date'])
497 prob_df['target_date']   = pd.to_datetime(prob_df['target_date'])
498
499 print()
500 print(f"Total records : {len(prob_df)}")
501 print(f"Models        : {prob_df['model'].unique().tolist()}")
502 print(f"Feature tiers : {prob_df['feature_tier'].unique().tolist()}")
503 prob_df.to_csv('results_probabilistic_NO1.csv', index=False)
504
505 # ============================================================================
506 # RESULT: CRPS Summary Table  (primary probabilistic result)
507 # ============================================================================
508
509 models   = list(prob_df['model'].unique())
510 horizons = sorted(prob_df['horizon'].unique())
511
512 rows = []
513 for m in models:
514     sub = prob_df[prob_df['model'] == m]
515     row = {'Model': m}
516     for h in horizons:
517         h_sub = sub[sub['horizon'] == h]
518         row[f'h={h}d'] = round(h_sub['crps'].mean(), 2) if len(h_sub) > 0
519         else np.nan
520     rows.append(row)
521 crps_table = pd.DataFrame(rows).set_index('Model')
522
523 print("="*65)
524 print("TABLE - CRPS by Model and Horizon (EUR/MWh, lower is better)")
525 print("Feature tiers: SHORT (h<=30d) | MEDIUM (h<=180d) | LONG (h>180d)")
526 print("="*65)
527 print(crps_table.to_string())
528 crps_table.to_csv('summary_crps_NO1.csv')
529 print("\nSaved: summary_crps_NO1.csv")
530
531 # CRPS improvement over Naive-Gaussian
532 print()
533 print("CRPS improvement over Naive-Gaussian (positive = better):")
534 if 'Naive-Gaussian' in crps_table.index:
```

```
    naive_row = crps_table.loc['Naive-Gaussian']
    for model in crps_table.index:
        if model == 'Naive-Gaussian':
            continue
        delta = ((naive_row - crps_table.loc[model]) / naive_row * 100).round(1)
        print(f"  {model}:")
        for col, d in delta.items():
            flag = "better" if d > 0 else "worse"
            print(f"    {col}: {'+' if d > 0 else ''}{d:.1f}% ({flag})")

# - Figure: CRPS by horizon ----------------------
fig, ax = plt.subplots(figsize=(12, 6))
fig.suptitle('CRPS by Forecasting Horizon\n'
             'Each model uses the horizon-appropriate feature set (SHORT / MEDIUM
             / LONG)', fontsize=13, fontweight='bold')

for model in models:
    sub  = prob_df[prob_df['model'] == model]
    vals = [sub[sub['horizon']==h]['crps'].mean() if (sub['horizon']==h).any()
            else np.nan for h in horizons]
    ax.plot(range(len(horizons)), vals,
            'o-', lw=2.5, ms=9,
            color=MODEL_COLORS.get(model, 'black'),
            label=model)

# Shade horizon zones
short_end  = next((i for i, h in enumerate(horizons) if h > 30),  len(horizons))
medium_end = next((i for i, h in enumerate(horizons) if h > 180), len(horizons))
ax.axvspan(-0.5,           short_end  - 0.5, alpha=0.07, color='steelblue')
ax.axvspan(short_end-0.5,  medium_end - 0.5, alpha=0.07, color='orange')
ax.axvspan(medium_end-0.5, len(horizons)-0.5, alpha=0.07, color='green')

ymax = ax.get_ylim()[1] if ax.get_ylim()[1] > 0 else 100
ax.text(max(short_end/2 - 0.5, 0),          ymax*0.95, 'SHORT\nFEATURES_SHORT',
        ha='center', fontsize=8.5, color='steelblue', fontweight='bold')
ax.text((short_end + medium_end)/2 - 0.5,   ymax*0.95, 'MEDIUM\nFEATURES_MEDIUM',
        ha='center', fontsize=8.5, color='darkorange', fontweight='bold')
ax.text((medium_end + len(horizons))/2 - 0.5, ymax*0.95, 'LONG\nFEATURES_LONG',
        ha='center', fontsize=8.5, color='darkgreen', fontweight='bold')

ax.set_xticks(range(len(horizons)))
ax.set_xticklabels([f'{h}d' for h in horizons])
ax.set_xlabel('Forecasting Horizon (days)')
ax.set_ylabel('Mean CRPS (EUR/MWh)')
ax.legend(fontsize=10)
plt.tight_layout()
plt.show()

# ============================================================================
# RESULT: Figure Fan Charts
# Shows actual prices vs prediction intervals at three key horizons.
# Each panel uses the feature set appropriate for that horizon.
# ============================================================================
def plot_fan(prob_df, model_name, horizons_to_plot, filename, suptitle):
    available = [h for h in horizons_to_plot if h in prob_df['horizon'].values
```

```
                and (prob_df['model'] == model_name).any()]
    if len(available) == 0:
        print(f"  No data for {model_name}")
        return

    fig, axes = plt.subplots(len(available), 1,
                             figsize=(15, 5 * len(available)),
                             gridspec_kw={'hspace': 0.55})
    if len(available) == 1:
        axes = [axes]

    fig.suptitle(suptitle, fontsize=12, fontweight='bold', y=0.995)
    color = MODEL_COLORS.get(model_name, 'steelblue')

    for ax, h in zip(axes, available):
        sub = (prob_df[(prob_df['model'] == model_name) & (prob_df['horizon']
        == h)]
               .sort_values('target_date').copy())
        if len(sub) == 0:
            continue

        tier  = sub['feature_tier'].iloc[0].upper() if 'feature_tier' in
        sub.columns else 'N/A'
        n     = len(sub)
        xi    = np.arange(n)
        dates = pd.to_datetime(sub['target_date'].values)

        def shade(lo_col, hi_col, alpha, label):
            if lo_col in sub.columns and hi_col in sub.columns:
                ax.fill_between(xi,
                                sub[lo_col].values.astype(float),
                                sub[hi_col].values.astype(float),
                                alpha=alpha, color=color, label=label)

        shade('q05', 'q95', 0.12, '95% PI')
        shade('q10', 'q90', 0.22, '80% PI')
        shade('q25', 'q75', 0.38, '50% PI')

        if 'q50' in sub.columns:
            ax.plot(xi, sub['q50'].values.astype(float),
                    color=color, lw=1.5, ls='--', alpha=0.9, label='Median (Q50)')

        ax.plot(xi, sub['y_true'].values.astype(float),
                color='black', lw=1.0, alpha=0.85, label='Actual price')

        # Shade crisis period
        idx_lo = np.searchsorted(dates, pd.Timestamp(VAL_START))
        idx_hi = np.searchsorted(dates, pd.Timestamp(VAL_END))
        if idx_lo < idx_hi < n:
            ax.axvspan(idx_lo, idx_hi, alpha=0.10, color='red',
            label='Crisis 2022')

        # Coverage annotation
        cov80_val = None
        for _, row in cov_df.iterrows():
```

```
            if (row['model'] == model_name and
                    row['horizon'] == h and
                    abs(row['nominal'] - 0.80) < 0.01):
                cov80_val = row['empirical']
                width_val = row['width']
                break

        if cov80_val is not None:
            ax.text(0.02, 0.95,
                    f"80% PI: empirical coverage = {cov80_val*100:.1f}%  "
                    f"(nominal = 80%)\nMean PI width = {width_val:.1f} EUR/MWh",
                    transform=ax.transAxes, fontsize=9, va='top',
                    bbox=dict(fc='white', ec='gray', alpha=0.85))


        step = max(1, n // 6)
        ax.set_xticks(xi[::step])
        ax.set_xticklabels([pd.Timestamp(dates[i]).strftime('%Y-%m')
                            for i in xi[::step]], rotation=30, ha='right',
                            fontsize=8)
        ax.set_title(f'h = {h} days  |  FEATURES_{tier}  '
                     f'({len(get_features(h))} features)',
                     fontsize=11)
        ax.set_ylabel('Spot Price (EUR/MWh)')
        ax.legend(fontsize=8, ncol=5, loc='upper right')

    plt.savefig(filename)
    plt.show()
    print(f"Saved: {filename}")


# - Figure: DMLP fan charts -------
print("Plotting DMLP fan charts (primary probabilistic figure)...")
plot_fan(
    prob_df,
    model_name='DMLP',
    horizons_to_plot=[7, 30, 90, 180],
    filename='fig_fan_charts_dmlp.png',
    suptitle=(
        'Fan Charts: DMLP\n'
        '50% / 80% / 95% Prediction Intervals vs Actual Prices\n'
    )
)

# - Figure: LASSO-QR fan charts ---
print("\nPlotting LASSO-QR fan charts (for comparison)...")
plot_fan(
    prob_df,
    model_name='LASSO-QR',
    horizons_to_plot=[7, 30, 90, 180],
    filename='fig_fan_charts_lasso.png',
    suptitle=(
        'Fan Charts: LASSO-QR\n'
        '50% / 80% / 95% Prediction Intervals vs Actual Prices\n'
    )
```

```
)

# - Print interval width comparison -------------------
print()
print("="*65)
print("Interval width comparison - why LASSO-QR fans are invisible")
print("="*65)
print()
print("80% Prediction Interval mean width (EUR/MWh):")
print()
w_table = cov_df[cov_df['nominal'] == 0.80].pivot_table(
    index='model', columns='horizon', values='width', aggfunc='first')
print(w_table.to_string())
print()

"""
Post-Estimation Calibration via Split Conformal Prediction
============================================================

Method: Split Conformal Prediction
Reference: Angelopoulos and Bates (2022), "A Gentle Introduction to
           Conformal Prediction"

How it works:
    1. Use the VALIDATION SET (2022) as the calibration set.
       This is data the model has already seen in evaluation but was
       never used for training - exactly what conformal prediction requires.
    2. Compute the non-conformity score for each calibration observation:
           s_i = max(q_lo_i - y_i,  y_i - q_hi_i)
       This measures how far the actual price falls outside the
       predicted interval. If y_i is inside the interval, s_i <= 0.
    3. For a desired coverage level (1 - alpha), compute the
       (1 - alpha)(1 + 1/n) quantile of the calibration scores.
       Call this q_hat.
    4. At test time, expand each predicted interval by q_hat:
           [q_lo - q_hat,  q_hi + q_hat]
       This guarantees marginal coverage of (1 - alpha) on the test set.
"""

import warnings; warnings.filterwarnings('ignore')
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import kstest

plt.rcParams.update({
    'figure.facecolor': 'white', 'axes.facecolor': '#f8f9fa',
    'axes.grid': True, 'grid.alpha': 0.35, 'font.size': 11,
    'axes.spines.top': False, 'axes.spines.right': False,
    'figure.dpi': 110, 'savefig.dpi': 150, 'savefig.bbox': 'tight',
})


def conformal_calibrate(prob_df, model_name,
                        cal_period_start, cal_period_end,
```

```
                        eval_period_start, eval_period_end,
                        levels=(0.50, 0.80, 0.90)):
    """
    Apply split conformal prediction to recalibrate prediction intervals.

    Parameters
    ----------
    prob_df           : full results DataFrame from rolling evaluation
    model_name        : 'DMLP' or 'LASSO-QR'
    cal_period_start  : start of calibration period (Timestamp)
    cal_period_end    : end of calibration period (Timestamp)
    eval_period_start : start of evaluation period (Timestamp)
    eval_period_end   : end of evaluation period (Timestamp)
    levels            : nominal coverage levels to calibrate

    Returns
    -------
    DataFrame: test-period predictions with original and calibrated intervals
    dict:      conformal quantile adjustments per (horizon, level)
    """
    model_df = prob_df[prob_df['model'] == model_name].copy()
    model_df['target_date'] = pd.to_datetime(model_df['target_date'])

    # Calibration set: validation period (2022 crisis)
    cal = model_df[
        (model_df['target_date'] >= cal_period_start) &
        (model_df['target_date'] <= cal_period_end)
    ].copy()

    # Test set: post-crisis period
    test = model_df[
        (model_df['target_date'] >= eval_period_start) &
        (model_df['target_date'] <= eval_period_end)
    ].copy()

    # Map nominal levels to stored quantile columns
    level_cols = {
        0.50: ('q25', 'q75'),
        0.80: ('q10', 'q90'),
        0.90: ('q05', 'q95'),
    }

    adjustments = {}   # (horizon, level) -> q_hat
    cal_coverage = {}  # (horizon, level) -> pre-calibration calibration-set
    coverage

    for h in sorted(model_df['horizon'].unique()):
        cal_h  = cal[cal['horizon'] == h]
        test_h = test[test['horizon'] == h]
        if len(cal_h) < 5 or len(test_h) < 5:
            continue

        for level in levels:
            if level not in level_cols:
                continue
```

```
            lo_col, hi_col = level_cols[level]
            if lo_col not in cal_h.columns or hi_col not in cal_h.columns:
                continue
            alpha = 1 - level

            # Non-conformity scores on calibration set
            lo_cal = cal_h[lo_col].values.astype(float)
            hi_cal = cal_h[hi_col].values.astype(float)
            y_cal  = cal_h['y_true'].values.astype(float)

            # Score: how far outside the interval is the actual price?
            # Negative means inside, positive means outside
            scores = np.maximum(lo_cal - y_cal, y_cal - hi_cal)

            # Conformal quantile: (1-alpha)(1+1/n) empirical quantile of scores
            n      = len(scores)
            q_level = np.ceil((1 - alpha) * (n + 1)) / n
            q_level = min(q_level, 1.0)
            q_hat   = float(np.quantile(scores, q_level))

            adjustments[(h, level)] = q_hat

            # Pre-calibration coverage on calibration set
            cal_cov = float(((y_cal >= lo_cal) & (y_cal <= hi_cal)).mean())
            cal_coverage[(h, level)] = cal_cov

    # Apply adjustments to test set
    test_calibrated = test.copy()
    for h in sorted(test['horizon'].unique()):
        mask = test_calibrated['horizon'] == h
        for level in levels:
            if (h, level) not in adjustments:
                continue
            lo_col, hi_col = level_cols[level]
            if lo_col not in test_calibrated.columns:
                continue
            q_hat = adjustments[(h, level)]
            # Expand interval symmetrically by q_hat
            test_calibrated.loc[mask, f'cal_{lo_col}'] = (
                test_calibrated.loc[mask, lo_col] - q_hat)
            test_calibrated.loc[mask, f'cal_{hi_col}'] = (
                test_calibrated.loc[mask, hi_col] + q_hat)

    return test_calibrated, adjustments, cal_coverage


# - Run conformal calibration -----------------------
print("Applying split conformal prediction to DMLP intervals...")
print("Calibration set: 2022 (validation / crisis period)")
print("Test set:        2023-2024")
print()

CAL_START  = pd.Timestamp('2022-01-01')
CAL_END    = pd.Timestamp('2022-12-31')
TEST_START_CAL = pd.Timestamp('2023-01-01')
```

```
TEST_END_CAL   = pd.Timestamp('2024-12-31')
LEVELS = [0.50, 0.80, 0.90]

dmlp_cal, adjustments, cal_cov = conformal_calibrate(
    prob_df, 'DMLP',
    CAL_START, CAL_END,
    TEST_START_CAL, TEST_END_CAL,
    levels=LEVELS
)

# - Print adjustment table -------------------------
print("Conformal adjustment (q_hat) per horizon and level:")
print("(interval expanded by this amount on each side, EUR/MWh)")
print()
horizons_avail = sorted(set(h for h, l in adjustments.keys()))
rows = []
for h in horizons_avail:
    row = {'Horizon': f'h={h}d'}
    for level in LEVELS:
        row[f'{int(level*100)}% PI'] = round(adjustments.get((h, level),
        np.nan), 2)
    rows.append(row)
adj_table = pd.DataFrame(rows).set_index('Horizon')
print(adj_table.to_string())

# - Coverage comparison: before vs after calibration -----------
print()
print("="*65)
print("COVERAGE COMPARISON - DMLP on TEST SET (2023-2024)")
print("80% Prediction Interval")
print("="*65)
print(f"{'Horizon':<10} {'Before calib':>14} {'After calib':>14}
{'Nominal':>10}")
print("-"*50)

level_cols = {0.50: ('q25','q75'), 0.80: ('q10','q90'), 0.90: ('q05','q95')}
coverage_results = []

for h in horizons_avail:
    test_h = dmlp_cal[dmlp_cal['horizon'] == h]
    if len(test_h) == 0:
        continue
    y = test_h['y_true'].values.astype(float)

    for level in LEVELS:
        lo_col, hi_col = level_cols[level]
        if lo_col not in test_h.columns:
            continue

        # Original coverage
        lo_orig = test_h[lo_col].values.astype(float)
        hi_orig = test_h[hi_col].values.astype(float)
        cov_orig = float(((y >= lo_orig) & (y <= hi_orig)).mean())

        # Calibrated coverage
```

```
        cal_lo_col = f'cal_{lo_col}'
        cal_hi_col = f'cal_{hi_col}'
        if cal_lo_col in test_h.columns:
            lo_cal = test_h[cal_lo_col].values.astype(float)
            hi_cal = test_h[cal_hi_col].values.astype(float)
            cov_cal = float(((y >= lo_cal) & (y <= hi_cal)).mean())
            width_cal = float((hi_cal - lo_cal).mean())
        else:
            cov_cal  = np.nan
            width_cal = np.nan

        coverage_results.append({
            'horizon': h, 'level': level,
            'before': round(cov_orig, 3),
            'after':  round(cov_cal, 3),
            'nominal': level,
            'width_after': round(width_cal, 1),
        })

        if level == 0.80:
            print(f"  h={h:3d}d   {cov_orig*100:>12.1f}%
            {cov_cal*100:>12.1f}%  "
                  f"{'80.0%':>10}")

cov_results_df = pd.DataFrame(coverage_results)
cov_results_df.to_csv('conformal_coverage_NO1.csv', index=False)
print()
print("Full results saved: conformal_coverage_NO1.csv")

# - Figure: Coverage before vs after calibration -------------
fig, axes = plt.subplots(3, 1, figsize=(10, 8), sharey=False)
fig.suptitle('Conformal Prediction Calibration of DMLP Intervals\n'
             'Calibration set: 2022  |  Test set: 2023-2024',
             fontsize=13, fontweight='bold')

nominal_colors = {0.50: '#9C27B0', 0.80: '#2196F3', 0.90: '#4CAF50'}
horizons_plot  = sorted(horizons_avail)
x = np.arange(len(horizons_plot))

for i, level in enumerate(LEVELS):
    ax = axes[i]
    sub = cov_results_df[cov_results_df['level'] == level]
    sub = sub.sort_values('horizon')

    before_vals = sub['before'].values * 100
    after_vals  = sub['after'].values  * 100

    ax.bar(x - 0.2, before_vals, 0.35,
           color='#FF9800', alpha=0.82, label='Before calibration')
    ax.bar(x + 0.2, after_vals,  0.35,
           color=nominal_colors[level], alpha=0.82, label='After calibration')
    ax.axhline(level * 100, color='black', ls='--', lw=2,
               label=f'Nominal {int(level*100)}%')

    ax.set_xticks(x)
```

```
    ax.set_xticklabels([f'h={h}d' for h in horizons_plot], fontsize=9)
    ax.set_ylabel('Empirical Coverage (%)')
    ax.set_title(f'{int(level*100)}% Prediction Interval')
    ax.set_ylim(0, 110)
    ax.legend(fontsize=8)

    # Annotation: improvement
    for j, (bv, av) in enumerate(zip(before_vals, after_vals)):
        diff = av - bv
        ax.text(j + 0.2, av + 2, f'+{diff:.0f}' if diff > 0 else f'{diff:.0f}',
                ha='center', fontsize=8,
                color='green' if diff > 0 else 'red')

plt.tight_layout()
plt.show()

print()
print("="*65)
print("SUMMARY")
print("="*65)
for level in [0.80]:
    print(f"\n{int(level*100)}% PI coverage improvement:")
    sub = cov_results_df[cov_results_df['level'] == level].sort_values('horizon')
    for _, row in sub.iterrows():
        improvement = (row['after'] - row['before']) * 100
        print(f"  h={int(row['horizon']):3d}d:  "
              f"{row['before']*100:.1f}% -> {row['after']*100:.1f}%  "
              f"(+{improvement:.1f} pp)  nominal=80%")

```